

LECTURE NOTES IN MACHINE LEARNING

Dr V N Krishnachandran

Vidya Centre for Artificial Intelligence Research
Vidya Academy of Science & Technology
Thrissur - 680501

Copyright © 2018 V. N. Krishnachandran

Published by
Vidya Centre for Artificial Intelligence Research
Vidya Academy of Science & Technology
Thrissur - 680501, Kerala, India

The book was typeset by the author using the L^AT_EX document preparation system.

Cover design: Author

Licensed under the Creative Commons Attribution 4.0 International (CC BY 4.0) License. You may not use this file except in compliance with the License. You may obtain a copy of the License at <https://creativecommons.org/licenses/by/4.0/>.

Price: Rs 0.00.

First printing: July 2018

Preface

The book is exactly what its title claims it to be: lecture notes; nothing more, nothing less!

A reader looking for elaborate descriptive expositions of the concepts and tools of machine learning will be disappointed with this book. There are plenty of books out there in the market with different styles of exposition. Some of them give a lot of emphasis on the mathematical theory behind the algorithms. In some others the emphasis is on the verbal descriptions of algorithms avoiding the use of mathematical notations and concepts to the maximum extent possible. There is one book the author of which is so afraid of introducing mathematical symbols that he introduces σ as “the Greek letter sigma similar to a b turned sideways”. But among these books, the author of these Notes could not spot a book that would give complete worked out examples illustrating the various algorithms. These notes are expected to fill this gap.

The focus of this book is on giving a quick and fast introduction to the basic concepts and important algorithms in machine learning. In nearly all cases, whenever a new concept is introduced it has been illustrated with “toy examples” and also with examples from real life situations. In the case of algorithms, wherever possible, the working of the algorithm has been illustrated with concrete numerical examples. In some cases, the full algorithm may contain heavy use of mathematical notations and concepts. Practitioners of machine learning sometimes treat such algorithms as “black box algorithms”. Student readers of this book may skip these details on a first reading.

The book is written primarily for the students pursuing the B Tech programme in Computer Science and Engineering of the APJ Abdul Kalam Technological University. The Curriculum for the programme offers a course on machine learning as an elective course in the Seventh Semester with code and name “CS 467 Machine Learning”. The selection of topics in the book was guided by the contents of the syllabus for the course. The book will also be useful to faculty members who teach the course.

Though the syllabus for CS 467 Machine Learning is reasonably well structured and covers most of the basic concepts of machine learning, there is some lack of clarity on the depth to which the various topics are to be covered. This ambiguity has been compounded by the lack of any mention of a single textbook for the course and unfortunately the books cited as references treat machine learning at varying levels. The guiding principle the author has adopted in the selection of materials in the preparation of these notes is that, at the end of the course, the student must acquire enough understanding about the methodologies and concepts underlying the various topics mentioned in the syllabus.

Any study of machine learning algorithms without studying their implementations in software packages is definitely incomplete. There are implementations of these algorithms available in the R and Python programming languages. Two or three lines of code may be sufficient to implement an algorithm. Since the syllabus for CS 467 Machine Learning does not mandate the study of such implementations, this aspect of machine learning has not been included in this book. The students are well advised to refer to any good book or the resources available in the internet to acquire a working knowledge of these implementations.

Evidently, there are no original material in this book. The readers can see shadows of everything presented here in other sources which include the reference books listed in the syllabus of the course referred to earlier, other books on machine learning, published research/review papers and also several open sources accessible through the internet. However, care has been taken to present the material borrowed from other sources in a format digestible to the targeted audience. There are

more than a hundred figures in the book. Nearly all of them were drawn using the TikZ package for L^AT_EX. A few of the figures were created using the R programming language. A small number of figures are reproductions of images available in various websites. There surely will be many errors – conceptual, technical and printing – in these notes. The readers are earnestly requested to point out such errors to the author so that an error free book can be brought up in the future.

The author wishes to put on record his thankfulness to Vidya Centre for Artificial Intelligence Research (V-CAIR) for agreeing to be the publisher of this book. V-CAIR is a research centre functioning in Vidya Academy of Science & Technology, Thrissur, Kerala, established as part of the “AI and Deep Learning: Skilling and Research” project launched by Royal Academy of Engineering, UK, in collaboration with University College, London, Brunel University, London and Bennett University, India.

VAST Campus
July 2018

Dr V N Krishnachandran
Department of Computer Applications
Vidya Academy of Science & Technology, Thrissur - 680501
(email: krishnachandran.vn@vidyaacademy.ac.in)

Syllabus

Course code	Course Name	L - T - P - Credits	Year of introduction
CS467	Machine Learning	3 - 0 - 0 - 3	2016

Course Objectives

- To introduce the prominent methods for machine learning
- To study the basics of supervised and unsupervised learning
- To study the basics of connectionist and other architectures

Syllabus

Introduction to Machine Learning, Learning in Artificial Neural Networks, Decision trees, HMM, SVM, and other Supervised and Unsupervised learning methods.

Expected Outcome

The students will be able to

- i) differentiate various learning approaches, and to interpret the concepts of supervised learning
- ii) compare the different dimensionality reduction techniques
- iii) apply theoretical foundations of decision trees to identify best split and Bayesian classifier to label data points
- iv) illustrate the working of classifier models like SVM, Neural Networks and identify classifier model for typical machine learning applications
- v) identify the state sequence and evaluate a sequence emission probability from a given HMM
- vi) illustrate and apply clustering algorithms and identify its applicability in real life problems

References

1. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
2. Ethem Alpaydin, *Introduction to Machine Learning* (Adaptive Computation and machine Learning), MIT Press, 2004.
3. Margaret H. Dunham, *Data Mining: Introductory and Advanced Topics*, Pearson, 2006.

4. Mitchell T., *Machine Learning*, McGraw Hill.
5. Ryszard S. Michalski, Jaime G. Carbonell, and Tom M. Mitchell, *Machine Learning : An Artificial Intelligence Approach*, Tioga Publishing Company.

Course Plan

Module I. Introduction to Machine Learning, Examples of Machine Learning applications - Learning associations, Classification, Regression, Unsupervised Learning, Reinforcement Learning. Supervised learning- Input representation, Hypothesis class, Version space, Vapnik-Chervonenkis (VC) Dimension

Hours: 6. Semester exam marks: 15%

Module II. Probably Approximately Learning (PAC), Noise, Learning Multiple classes, Model Selection and Generalization, Dimensionality reduction- Subset selection, Principle Component Analysis

Hours: 8. Semester exam marks: 15%

FIRST INTERNAL EXAMINATION

Module III. Classification- Cross validation and re-sampling methods- Kfold cross validation, Boot strapping, Measuring classifier performance- Precision, recall, ROC curves. Bayes Theorem, Bayesian classifier, Maximum Likelihood estimation, Density functions, Regression

Hours: 8. Semester exam marks: 20%

Module IV. Decision Trees- Entropy, Information Gain, Tree construction, ID3, Issues in Decision Tree learning- Avoiding Over-fitting, Reduced Error Pruning, The problem of Missing Attributes, Gain Ratio, Classification by Regression (CART), Neural Networks- The Perceptron, Activation Functions, Training Feed Forward Network by Back Propagation.

Hours: 6. Semester exam marks: 15%

SECOND INTERNAL EXAMINATION

Module V. Kernel Machines - Support Vector Machine - Optimal Separating hyper plane, Soft-margin hyperplane, Kernel trick, Kernel functions. Discrete Markov Processes, Hidden Markov models, Three basic problems of HMMs - Evaluation problem, finding state sequence, Learning model parameters. Combining multiple learners, Ways to achieve diversity, Model combination schemes, Voting, Bagging, Booting

Hours: 8. Semester exam marks: 20%

Module VI. Unsupervised Learning - Clustering Methods - K-means, Expect-ation-Maxi-mization Algorithm, Hierarchical Clustering Methods, Density based clustering

Hours: 6. Semester exam marks: 15%

END SEMESTER EXAMINATION

Question paper pattern

1. There will be FOUR parts in the question paper: A, B, C, D.

2. Part A

- a) Total marks: 40
- b) TEN questions, each have 4 marks, covering all the SIX modules (THREE questions from modules I & II; THREE questions from modules III & IV; FOUR questions from modules V & VI).

- c) All the TEN questions have to be answered.

3. Part B

- a) Total marks: 18
- b) THREE questions, each having 9 marks. One question is from module I; one question is from module II; one question uniformly covers modules I & II.
- c) Any TWO questions have to be answered.
- d) Each question can have maximum THREE subparts.

4. Part C

- a) Total marks: 18
- b) THREE questions, each having 9 marks. One question is from module III; one question is from module IV; one question uniformly covers modules III & IV.
- c) Any TWO questions have to be answered.
- d) Each question can have maximum THREE subparts.

5. Part D

- a) Total marks: 24
- b) THREE questions, each having 12 marks. One question is from module V; one question is from module VI; one question uniformly covers modules V & VI.
- c) Any TWO questions have to be answered.
- d) Each question can have maximum THREE subparts.

6. There will be AT LEAST 60% analytical/numerical questions in all possible combinations of question choices.

Contents

Introduction	iii
Syllabus	v
1 Introduction to machine learning	1
1.1 Introduction	1
1.2 How machines learn	2
1.3 Applications of machine learning	3
1.4 Understanding data	4
1.5 General classes of machine learning problems	6
1.6 Different types of learning	11
1.7 Sample questions	13
2 Some general concepts	15
2.1 Input representation	15
2.2 Hypothesis space	15
2.3 Ordering of hypotheses	18
2.4 Version space	19
2.5 Noise	22
2.6 Learning multiple classes	22
2.7 Model selection	23
2.8 Generalisation	24
2.9 Sample questions	25
3 VC dimension and PAC learning	27
3.1 Vapnik-Chervonenkis dimension	27
3.2 Probably approximately correct learning	31
3.3 Sample questions	34
4 Dimensionality reduction	35
4.1 Introduction	35
4.2 Why dimensionality reduction is useful	36
4.3 Subset selection	36
4.4 Principal component analysis	38
4.5 Sample questions	46
5 Evaluation of classifiers	48
5.1 Methods of evaluation	48
5.2 Cross-validation	49
5.3 K -fold cross-validation	49
5.4 Measuring error	51
5.5 Receiver Operating Characteristic (ROC)	54
5.6 Sample questions	58

6	Bayesian classifier and ML estimation	61
6.1	Conditional probability	61
6.2	Bayes' theorem	62
6.3	Naive Bayes algorithm	64
6.4	Using numeric features with naive Bayes algorithm	67
6.5	Maximum likelihood estimation (ML estimation)	68
6.6	Sample questions	70
7	Regression	72
7.1	Definition	72
7.2	Criterion for minimisation of error	73
7.3	Simple linear regression	74
7.4	Polynomial regression	77
7.5	Multiple linear regression	78
7.6	Sample questions	80
8	Decision trees	83
8.1	Decision tree: Example	83
8.2	Two types of decision trees	84
8.3	Classification trees	84
8.4	Feature selection measures	89
8.5	Entropy	89
8.6	Information gain	92
8.7	Gini indices	93
8.8	Gain ratio	94
8.9	Decision tree algorithms	95
8.10	The ID3 algorithm	96
8.11	Regression trees	101
8.12	CART algorithm	105
8.13	Other decision tree algorithms	105
8.14	Issues in decision tree learning	106
8.15	Avoiding overfitting of data	106
8.16	Problem of missing attributes	107
8.17	Sample questions	108
9	Neural networks	111
9.1	Introduction	111
9.2	Biological motivation	111
9.3	Artificial neurons	112
9.4	Activation function	113
9.5	Perceptron	116
9.6	Artificial neural networks	119
9.7	Characteristics of an ANN	119
9.8	Backpropagation	122
9.9	Introduction to deep learning	129
9.10	Sample questions	131
10	Support vector machines	133
10.1	An example	133
10.2	Finite dimensional vector spaces	138
10.3	Hyperplanes	141
10.4	Two-class data sets	144
10.5	Linearly separable data	144
10.6	Maximal margin hyperplanes	145
10.7	Mathematical formulation of the SVM problem	147

10.8	Solution of the SVM problem	149
10.9	Soft margin hyperlanes	154
10.10	Kernel functions	155
10.11	The kernel method (kernel trick)	157
10.12	Multiclass SVM's	158
10.13	Sample questions	159
11	Hidden Markov models	161
11.1	Discrete Markov processes: Examples	161
11.2	Discrete Markov processes: General case	163
11.3	Hidden Markov models	167
11.4	Three basic problems of HMMs	169
11.5	HMM application: Isolated word recognition	170
11.6	Sample questions	171
12	Combining multiple learners	173
12.1	Why combine many learners	173
12.2	Ways to achieve diversity	173
12.3	Model combination schemes	174
12.4	Ensemble learning*	176
12.5	Random forest*	176
12.6	Sample questions	178
13	Clustering methods	179
13.1	Clustering	179
13.2	k -means clustering	179
13.3	Multi-modal distributions	186
13.4	Mixture of normal distributions	186
13.5	Mixtures in terms of latent variables	188
13.6	Expectation-maximisation algorithm	189
13.7	The EM algorithm for Gaussian mixtures	190
13.8	Hierarchical clustering	191
13.9	Measures of dissimilarity	194
13.10	Algorithm for agglomerative hierarchical clustering	196
13.11	Algorithm for divisive hierarchical clustering	200
13.12	Density-based clustering	203
13.13	Sample questions	204
	Bibliography	206
	Index	207

List of Figures

1.1	Components of learning process	2
1.2	Example for “examples” and “features” collected in a matrix format (data relates to automobiles and their features)	5
1.3	Graphical representation of data in Table 1.1. Solid dots represent data in “Pass” class and hollow dots data in “Fail” class. The class label of the square dot is to be determined.	7
1.4	Supervised learning	12
2.1	Data in Table 2.1 with hollow dots representing positive examples and solid dots representing negative examples	16
2.2	An example hypothesis defined by Eq. (2.5)	17
2.3	Hypothesis h' is more general than hypothesis h'' if and only if $S'' \subseteq S'$	18
2.4	Values of m which define the version space with data in Table 2.1 and hypothesis space defined by Eq.(2.4)	19
2.5	Scatter plot of price-power data (hollow circles indicate positive examples and solid dots indicate negative examples)	20
2.6	The version space consists of hypotheses corresponding to axis-aligned rectangles contained in the shaded region	20
2.7	Examples for overfitting and overfitting models	24
2.8	Fitting a classification boundary	25
3.1	Different forms of the set $\{x \in S : h(x) = 1\}$ for $D = \{a, b, c\}$	28
3.2	Geometrical representation of the hypothesis $h_{a,b,c}$	30
3.3	A hypothesis $h_{a,b,c}$ consistent with the dichotomy defined by the subset $\{A, C\}$ of $\{A, B, C\}$	30
3.4	There is no hypothesis $h_{a,b,c}$ consistent with the dichotomy defined by the subset $\{A, C\}$ of $\{A, B, C, D\}$	30
3.5	An axis-aligned rectangle in the Euclidean plane	32
3.6	Axis-aligned rectangle which gives the tightest fit to the positive examples	33
4.1	Principal components	39
4.2	Scatter plot of data in Table 4.2	43
4.3	Coordinate system for principal components	45
4.4	Projections of data points on the axis of the first principal component	46
4.5	Geometrical representation of one-dimensional approximation to the data in Table 4.2	46
5.1	One iteration in a 5-fold cross-validation	50
5.2	The ROC space and some special points in the space	56
5.3	ROC curves of three different classifiers A, B, C	57
5.4	ROC curve of data in Table 5.3 showing the points closest to the perfect prediction point $(0, 1)$	58
6.1	Events A, B, C which are not mutually independent: Eqs.(6.1)–(6.3) are satisfied, but Eq.(6.4) is not satisfied.	62

6.2	Events A, B, C which are not mutually independent: Eq.(6.4) is satisfied but Eqs.(6.1)–(6.2) are not satisfied.	62
6.3	Discretization of numeric data: Example	68
7.1	Errors in observed values	74
7.2	Regression model for Table 7.2	76
7.3	Plot of quadratic polynomial model	78
7.4	The regression plane for the data in Table 7.4	80
8.1	Example for a decision tree	83
8.2	The graph-theoretical representation of the decision tree in Figure 8.6	84
8.3	Classification tree	85
8.4	Classification tree	86
8.5	Classification tree	88
8.6	Plot of p vs. Entropy	90
8.7	Root node of the decision tree for data in Table 8.9	97
8.8	Decision tree for data in Table 8.9, after selecting the branching feature at root node	99
8.9	Decision tree for data in Table 8.9, after selecting the branching feature at Node 1	100
8.10	Decision tree for data in Table 8.9	101
8.11	Part of a regression tree for Table 8.11	102
8.12	Part of regression tree for Table 8.11	102
8.13	A regression tree for Table 8.11	103
8.14	Impact of overfitting in decision tree learning	107
9.1	Anatomy of a neuron	111
9.2	Flow of signals in a biological neuron	112
9.3	Schematic representation of an artificial neuron	112
9.4	Simplified representation of an artificial neuron	113
9.5	Threshold activation function	114
9.6	Unit step activation function	114
9.7	The sigmoid activation function	114
9.8	Linear activation function	115
9.9	Piecewise linear activation function	115
9.10	Gaussian activation function	115
9.11	Hyperbolic tangent activation function	116
9.12	Schematic representation of a perceptron	116
9.13	Representation of x_1 AND x_2 by a perceptron	117
9.14	An ANN with only one layer	120
9.15	An ANN with two layers	121
9.16	Examples of different topologies of networks	122
9.17	A simplified model of the error surface showing the direction of gradient	123
9.18	ANN for illustrating backpropagation algorithm	124
9.19	ANN for illustrating backpropagation algorithm with initial values for weights	124
9.20	Notations of backpropagation algorithm	128
9.21	Notations of backpropagation algorithm: The i -th node in layer j	128
9.22	A shallow neural network	130
9.23	A deep neural network with three hidden layers	130
10.1	Scatter plot of data in Table 10.1 (filled circles represent “yes” and unfilled circles “no”)	134
10.2	Scatter plot of data in Table 10.1 with a separating line	135
10.3	Two separating lines for the data in Table 10.1	135
10.4	Shortest perpendicular distance of a separating line from data points	136
10.5	Maximum margin line for data in Table 10.1	136
10.6	Support vectors for data in Table 10.1	137

10.7	Boundaries of “street” of maximum width separating “yes” points and “no” points in Table 10.1	137
10.8	Plot of the maximum margin line of data in Table 10.1 produced by the R programming language	138
10.9	Half planes defined by a line	142
10.10	Perpendicular distance of a point from a plane	143
10.11	Scatterplot of data in Table 10.2	145
10.12	Maximal separating hyperplane, margin and support vectors	146
10.13	Maximal margin hyperplane of a 2-sample set in 2-dimensional space	147
10.14	Maximal margin hyperplane of a 3-sample set in 2-dimensional space	147
10.15	Soft margin hyperplanes	155
10.16	One-against all	158
10.17	One-against-one	159
11.1	A state diagram showing state transition probabilities	162
11.2	A two-coin model of an HMM	167
11.3	An N -state urn and ball model which illustrates the general case of a discrete symbol HMM	168
11.4	Block diagram of an isolated word HMM recogniser	171
12.1	Example of random forest with majority voting	177
13.1	Scatter diagram of data in Table 13.1	180
13.2	Initial choice of cluster centres and the resulting clusters	181
13.3	Cluster centres after first iteration and the corresponding clusters	182
13.4	New cluster centres and the corresponding clusters	183
13.5	Probability distributions	186
13.6	Graph of pdf defined by Eq.(13.9) superimposed on the histogram of the data in Table 13.3	188
13.7	A dendrogram of the dataset $\{a, b, c, d, e\}$	192
13.8	Different ways of drawing dendrogram	192
13.9	A dendrogram of the dataset $\{a, b, c, d, e\}$ showing the distances (heights) of the clusters at different levels	192
13.10	Hierarchical clustering using agglomerative method	193
13.11	Hierarchical clustering using divisive method	195
13.12	Length of the solid line “ ae ” is $\max\{d(x, y) : x \in A, y \in B\}$	196
13.13	Length of the solid line “ bc ” is $\min\{d(x, y) : x \in A, y \in B\}$	196
13.14	Dendrogram for the data given in Table 13.4 (complete linkage clustering)	199
13.15	Dendrogram for the data given in Table 13.4 (single linkage clustering)	200
13.16	$D_x = (\text{average of dashed lines}) - (\text{average of solid lines})$	201
13.17	Clusters of points and noise points not belonging to any of those clusters	203
13.18	With $m_0 = 4$: (a) p a point of high density (b) p a core point (c) p a border point (d) r a noise point	203
13.19	With $m_0 = 4$: (a) q is directly density-reachable from p (b) q is indirectly density-reachable from p	204

Chapter 1

Introduction to machine learning

In this chapter, we consider different definitions of the term “machine learning” and explain what is meant by “learning” in the context of machine learning. We also discuss the various components of the machine learning process. There are also brief discussions about different types learning like supervised learning, unsupervised learning and reinforcement learning.

1.1 Introduction

1.1.1 Definition of machine learning

Arthur Samuel, an early American leader in the field of computer gaming and artificial intelligence, coined the term “Machine Learning” in 1959 while at IBM. He defined machine learning as “the field of study that gives computers the ability to learn without being explicitly programmed.” However, there is no universally accepted definition for machine learning. Different authors define the term differently. We give below two more definitions.

1. Machine learning is programming computers to optimize a performance criterion using example data or past experience. We have a model defined up to some parameters, and learning is the execution of a computer program to optimize the parameters of the model using the training data or past experience. The model may be predictive to make predictions in the future, or descriptive to gain knowledge from data, or both (see [2] p.3).
2. The field of study known as machine learning is concerned with the question of how to construct computer programs that automatically improve with experience (see [4], Preface.).

Remarks

In the above definitions we have used the term “model” and we will be using this term at several contexts later in this book. It appears that there is no universally accepted one sentence definition of this term. Loosely, it may be understood as some mathematical expression or equation, or some mathematical structures such as graphs and trees, or a division of sets into disjoint subsets, or a set of logical “if ... then ... else ...” rules, or some such thing. It may be noted that this is not an exhaustive list.

1.1.2 Definition of learning

Definition

A computer program is said to *learn* from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks T , as measured by P , improves with experience E .

Examples

- i) Handwriting recognition learning problem
 - Task T : Recognising and classifying handwritten words within images
 - Performance P : Percent of words correctly classified
 - Training experience E : A dataset of handwritten words with given classifications
- ii) A robot driving learning problem
 - Task T : Driving on highways using vision sensors
 - Performance measure P : Average distance traveled before an error
 - training experience: A sequence of images and steering commands recorded while observing a human driver
- iii) A chess learning problem
 - Task T : Playing chess
 - Performance measure P : Percent of games won against opponents
 - Training experience E : Playing practice games against itself

Definition

A computer program which learns from experience is called a *machine learning program* or simply a *learning program*. Such a program is sometimes also referred to as a *learner*.

1.2 How machines learn**1.2.1 Basic components of learning process**

The learning process, whether by a human or a machine, can be divided into four components, namely, data storage, abstraction, generalization and evaluation. Figure 1.1 illustrates the various components and the steps involved in the learning process.

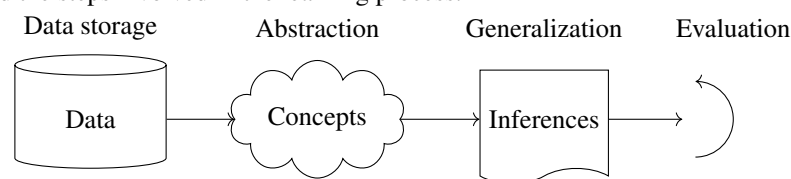


Figure 1.1: Components of learning process

1. Data storage

Facilities for storing and retrieving huge amounts of data are an important component of the learning process. Humans and computers alike utilize data storage as a foundation for advanced reasoning.

- In a human being, the data is stored in the brain and data is retrieved using electrochemical signals.
- Computers use hard disk drives, flash memory, random access memory and similar devices to store data and use cables and other technology to retrieve data.

2. Abstraction

The second component of the learning process is known as *abstraction*.

Abstraction is the process of extracting knowledge about stored data. This involves creating general concepts about the data as a whole. The creation of knowledge involves application of known models and creation of new models.

The process of fitting a model to a dataset is known as *training*. When the model has been trained, the data is transformed into an abstract form that summarizes the original information.

3. Generalization

The third component of the learning process is known as *generalisation*.

The term generalization describes the process of turning the knowledge about stored data into a form that can be utilized for future action. These actions are to be carried out on tasks that are similar, but not identical, to those what have been seen before. In generalization, the goal is to discover those properties of the data that will be most relevant to future tasks.

4. Evaluation

Evaluation is the last component of the learning process.

It is the process of giving feedback to the user to measure the utility of the learned knowledge. This feedback is then utilised to effect improvements in the whole learning process.

1.3 Applications of machine learning

Application of machine learning methods to large databases is called data mining. In data mining, a large volume of data is processed to construct a simple model with valuable use, for example, having high predictive accuracy.

The following is a list of some of the typical applications of machine learning.

1. In retail business, machine learning is used to study consumer behaviour.
2. In finance, banks analyze their past data to build models to use in credit applications, fraud detection, and the stock market.
3. In manufacturing, learning models are used for optimization, control, and troubleshooting.
4. In medicine, learning programs are used for medical diagnosis.
5. In telecommunications, call patterns are analyzed for network optimization and maximizing the quality of service.
6. In science, large amounts of data in physics, astronomy, and biology can only be analyzed fast enough by computers. The World Wide Web is huge; it is constantly growing and searching for relevant information cannot be done manually.
7. In artificial intelligence, it is used to teach a system to learn and adapt to changes so that the system designer need not foresee and provide solutions for all possible situations.
8. It is used to find solutions to many problems in vision, speech recognition, and robotics.
9. Machine learning methods are applied in the design of computer-controlled vehicles to steer correctly when driving on a variety of roads.
10. Machine learning methods have been used to develop programmes for playing games such as chess, backgammon and Go.

1.4 Understanding data

Since an important component of the machine learning process is data storage, we briefly consider in this section the different types and forms of data that are encountered in the machine learning process.

1.4.1 Unit of observation

By a *unit of observation* we mean the smallest entity with measured properties of interest for a study.

Examples

- A person, an object or a thing
- A time point
- A geographic region
- A measurement

Sometimes, units of observation are combined to form units such as person-years.

1.4.2 Examples and features

Datasets that store the units of observation and their properties can be imagined as collections of data consisting of the following:

- **Examples**

An “example” is an instance of the unit of observation for which properties have been recorded. An “example” is also referred to as an “instance”, or “case” or “record.” (It may be noted that the word “example” has been used here in a technical sense.)

- **Features**

A “feature” is a recorded property or a characteristic of examples. It is also referred to as “attribute”, or “variable” or “feature.”

Examples for “examples” and “features”

1. Cancer detection

Consider the problem of developing an algorithm for detecting cancer. In this study we note the following.

- (a) The units of observation are the patients.
- (b) The examples are members of a sample of cancer patients.
- (c) The following attributes of the patients may be chosen as the features:
 - gender
 - age
 - blood pressure
 - the findings of the pathology report after a biopsy

2. Pet selection

Suppose we want to predict the type of pet a person will choose.

- (a) The units are the persons.
- (b) The examples are members of a sample of persons who own pets.

year	model	price	mileage	color	transmission
2011	SEL	21992	7413	Yellow	AUTO
2011	SEL	20995	10926	Gray	AUTO
2011	SEL	19995	7351	Silver	AUTO
2011	SEL	17809	11613	Gray	AUTO
2012	SE	17500	8367	White	MANUAL
2010	SEL	17495	25125	Silver	AUTO
2011	SEL	17000	27393	Blue	AUTO
2010	SEL	16995	21026	Silver	AUTO
2011	SES	16995	32655	Silver	AUTO

Figure 1.2: Example for “examples” and “features” collected in a matrix format (data relates to automobiles and their features)

- (c) The features might include age, home region, family income, etc. of persons who own pets.

3. Spam e-mail

Let it be required to build a learning algorithm to identify spam e-mail.

- (a) The unit of observation could be an e-mail messages.
 (b) The examples would be specific messages.
 (c) The features might consist of the words used in the messages.

Examples and features are generally collected in a “matrix format”. Fig. 1.2 shows such a data set.

1.4.3 Different forms of data

1. Numeric data

If a feature represents a characteristic measured in numbers, it is called a numeric feature.

2. Categorical or nominal

A categorical feature is an attribute that can take on one of a limited, and usually fixed, number of possible values on the basis of some qualitative property. A categorical feature is also called a nominal feature.

3. Ordinal data

This denotes a nominal variable with categories falling in an ordered list. Examples include clothing sizes such as small, medium, and large, or a measurement of customer satisfaction on a scale from “not at all happy” to “very happy.”

Examples

In the data given in Fig.1.2, the features “year”, “price” and “mileage” are numeric and the features “model”, “color” and “transmission” are categorical.

1.5 General classes of machine learning problems

1.5.1 Learning associations

1. Association rule learning

Association rule learning is a machine learning method for discovering interesting relations, called “association rules”, between variables in large databases using some measures of “interestingness”.

2. Example

Consider a supermarket chain. The management of the chain is interested in knowing whether there are any patterns in the purchases of products by customers like the following:

“If a customer buys onions and potatoes together, then he/she is likely to also buy hamburger.”

From the standpoint of customer behaviour, this defines an association between the set of products {onion, potato} and the set {burger}. This association is represented in the form of a rule as follows:

$$\{\text{onion, potato}\} \Rightarrow \{\text{burger}\}$$

The measure of how likely a customer, who has bought onion and potato, to buy burger also is given by the conditional probability

$$P(\{\text{onion, potato}\}|\{\text{burger}\}).$$

If this conditional probability is 0.8, then the rule may be stated more precisely as follows:

“80% of customers who buy onion and potato also buy burger.”

3. How association rules are made use of

Consider an association rule of the form

$$X \Rightarrow Y,$$

that is, if people buy X then they are also likely to buy Y .

Suppose there is a customer who buys X and does not buy Y . Then that customer is a potential Y customer. Once we find such customers, we can target them for cross-selling. A knowledge of such rules can be used for promotional pricing or product placements.

4. General case

In finding an association rule $X \Rightarrow Y$, we are interested in learning a conditional probability of the form $P(Y|X)$ where Y is the product the customer may buy and X is the product or the set of products the customer has already purchased.

If we may want to make a distinction among customers, we may estimate $P(Y|X, D)$ where D is a set of customer attributes, like gender, age, marital status, and so on, assuming that we have access to this information.

5. Algorithms

There are several algorithms for generating association rules. Some of the well-known algorithms are listed below:

- a) Apriori algorithm
- b) Eclat algorithm
- c) FP-Growth Algorithm (FP stands for Frequency Pattern)

1.5.2 Classification

1. Definition

In machine learning, *classification* is the problem of identifying to which of a set of categories a new observation belongs, on the basis of a training set of data containing observations (or instances) whose category membership is known.

2. Example

Consider the following data:

Score1	29	22	10	31	17	33	32	20
Score2	43	29	47	55	18	54	40	41
Result	Pass	Fail	Fail	Pass	Fail	Pass	Pass	Pass

Table 1.1: Example data for a classification problem

Data in Table 1.1 is the training set of data. There are two attributes “Score1” and “Score2”. The class label is called “Result”. The class label has two possible values “Pass” and “Fail”. The data can be divided into two categories or classes: The set of data for which the class label is “Pass” and the set of data for which the class label is “Fail”.

Let us assume that we have no knowledge about the data other than what is given in the table. Now, the problem can be posed as follows: If we have some new data, say “Score1 = 25” and “Score2 = 36”, what value should be assigned to “Result” corresponding to the new data; in other words, to which of the two categories or classes the new observation should be assigned? See Figure 1.3 for a graphical representation of the problem.

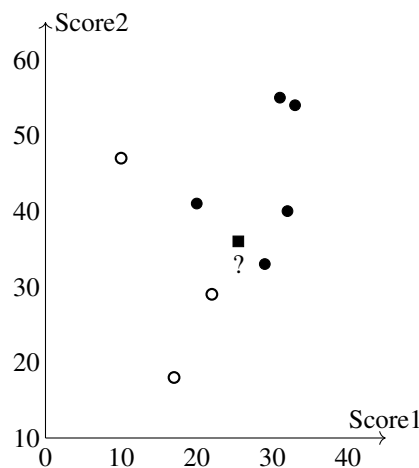


Figure 1.3: Graphical representation of data in Table 1.1. Solid dots represent data in “Pass” class and hollow dots data in “Fail” class. The class label of the square dot is to be determined.

To answer this question, using the given data alone we need to find the rule, or the formula, or the method that has been used in assigning the values to the class label “Result”. The problem of finding this rule or formula or the method is the classification problem. In general, even the general form of the rule or function or method will not be known. So several different rules, etc. may have to be tested to obtain the correct rule or function or method.

3. Real life examples

i) **Optical character recognition**

Optical character recognition problem, which is the problem of recognizing character codes from their images, is an example of classification problem. This is an example where there are multiple classes, as many as there are characters we would like to recognize. Especially interesting is the case when the characters are handwritten. People have different handwriting styles; characters may be written small or large, slanted, with a pen or pencil, and there are many possible images corresponding to the same character.

ii) **Face recognition**

In the case of *face recognition*, the input is an image, the classes are people to be recognized, and the learning program should learn to associate the face images to identities. This problem is more difficult than optical character recognition because there are more classes, input image is larger, and a face is three-dimensional and differences in pose and lighting cause significant changes in the image.

iii) **Speech recognition**

In *speech recognition*, the input is acoustic and the classes are words that can be uttered.

iv) **Medical diagnosis**

In *medical diagnosis*, the inputs are the relevant information we have about the patient and the classes are the illnesses. The inputs contain the patient's age, gender, past medical history, and current symptoms. Some tests may not have been applied to the patient, and thus these inputs would be missing.

v) **Knowledge extraction**

Classification rules can also be used for *knowledge extraction*. The rule is a simple model that explains the data, and looking at this model we have an explanation about the process underlying the data.

vi) **Compression**

Classification rules can be used for *compression*. By fitting a rule to the data, we get an explanation that is simpler than the data, requiring less memory to store and less computation to process.

vii) **More examples**

Here are some further examples of classification problems.

- (a) An emergency room in a hospital measures 17 variables like blood pressure, age, etc. of newly admitted patients. A decision has to be made whether to put the patient in an ICU. Due to the high cost of ICU, only patients who may survive a month or more are given higher priority. Such patients are labeled as "low-risk patients" and others are labeled "high-risk patients". The problem is to devise a rule to classify a patient as a "low-risk patient" or a "high-risk patient".
- (b) A credit card company receives hundreds of thousands of applications for new cards. The applications contain information regarding several attributes like annual salary, age, etc. The problem is to devise a rule to classify the applicants to those who are credit-worthy, who are not credit-worthy or to those who require further analysis.
- (c) Astronomers have been cataloguing distant objects in the sky using digital images created using special devices. The objects are to be labeled as star, galaxy, nebula, etc. The data is highly noisy and are very faint. The problem is to devise a rule using which a distant object can be correctly labeled.

4. Discriminant

A *discriminant* of a classification problem is a rule or a function that is used to assign labels to new observations.

Examples

- i) Consider the data given in Table 1.1 and the associated classification problem. We may consider the following rules for the classification of the new data:

IF Score1 + Score2 $\geq$ 60, THEN “Pass” ELSE “Fail”.
IF Score1 $\geq$ 20 AND Score2 $\geq$ 40 THEN “Pass” ELSE “Fail”.

Or, we may consider the following rules with unspecified values for M, m_1, m_2 and then by some method estimate their values.

IF Score1 + Score2 $\geq$ M , THEN “Pass” ELSE “Fail”.
IF Score1 $\geq$ m_1 AND Score2 $\geq$ m_2 THEN “Pass” ELSE “Fail”.

- ii) Consider a finance company which lends money to customers. Before lending money, the company would like to assess the risk associated with the loan. For simplicity, let us assume that the company assesses the risk based on two variables, namely, the annual income and the annual savings of the customers.

Let x_1 be the annual income and x_2 be the annual savings of a customer.

- After using the past data, a rule of the following form with suitable values for θ_1 and θ_2 may be formulated:

IF $x_1 > \theta_1$ AND $x_2 > \theta_2$ THEN “low-risk” ELSE “high-risk”.

This rule is an example of a discriminant.

- Based on the past data, a rule of the following form may also be formulated:

IF $x_2 - 0.2x_1 > 0$ THEN “low-risk” ELSE “high-risk”.

In this case the rule may be thought of as the discriminant. The function $f(x_1, x_2) = x_2 - 0.2x_1$ can also be considered as the discriminant.

5. Algorithms

There are several machine learning algorithms for classification. The following are some of the well-known algorithms.

- Logistic regression
- Naive Bayes algorithm
- k -NN algorithm
- Decision tree algorithm
- Support vector machine algorithm
- Random forest algorithm

Remarks

- A classification problem requires that examples be classified into one of two or more classes.
- A classification can have real-valued or discrete input variables.
- A problem with two classes is often called a two-class or binary classification problem.
- A problem with more than two classes is often called a multi-class classification problem.
- A problem where an example is assigned multiple classes is called a multi-label classification problem.

1.5.3 Regression**1. Definition**

In machine learning, a *regression problem* is the problem of predicting the value of a numeric variable based on observed values of the variable. The value of the output variable may be a number, such as an integer or a floating point value. These are often quantities, such as amounts and sizes. The input variables may be discrete or real-valued.

2. Example

Consider the data on car prices given in Table 1.2.

Price (US\$)	Age (years)	Distance (KM)	Weight (pounds)
13500	23	46986	1165
13750	23	72937	1165
13950	24	41711	1165
14950	26	48000	1165
13750	30	38500	1170
12950	32	61000	1170
16900	27	94612	1245
18600	30	75889	1245
21500	27	19700	1185
12950	23	71138	1105

Table 1.2: Prices of used cars: example data for regression

Suppose we are required to estimate the price of a car aged 25 years with distance 53240 KM and weight 1200 pounds. This is an example of a regression problem because we have to predict the value of the numeric variable “Price”.

3. General approach

Let x denote the set of input variables and y the output variable. In machine learning, the general approach to regression is to assume a model, that is, some mathematical relation between x and y , involving some parameters say, θ , in the following form:

$$y = f(x, \theta)$$

The function $f(x, \theta)$ is called the *regression function*. The machine learning algorithm optimizes the parameters in the set θ such that the approximation error is minimized; that is, the estimates of the values of the dependent variable y are as close as possible to the correct values given in the training set.

Example

For example, if the input variables are “Age”, “Distance” and “Weight” and the output variable is “Price”, the model may be

$$y = f(x, \theta)$$

$$\text{Price} = a_0 + a_1 \times (\text{Age}) + a_2 \times (\text{Distance}) + a_3 \times (\text{Weight})$$

where $x = (\text{Age}, \text{Distance}, \text{Weight})$ denotes the the set of input variables and $\theta = (a_0, a_1, a_2, a_3)$ denotes the set of parameters of the model.

4. Different regression models

There are various types of regression techniques available to make predictions. These techniques mostly differ in three aspects, namely, the number and type of independent variables, the type of dependent variables and the shape of regression line. Some of these are listed below.

- *Simple linear regression*: There is only one continuous independent variable x and the assumed relation between the independent variable and the dependent variable y is

$$y = a + bx.$$

- *Multivariate linear regression*: There are more than one independent variable, say $x_1, \dots, x_n$, and the assumed relation between the independent variables and the dependent variable is

$$y = a_0 + a_1x_1 + \dots + a_nx_n.$$

- *Polynomial regression*: There is only one continuous independent variable x and the assumed model is

$$y = a_0 + a_1x + \dots + a_nx^n.$$

- *Logistic regression*: The dependent variable is binary, that is, a variable which takes only the values 0 and 1. The assumed model involves certain probability distributions.

1.6 Different types of learning

In general, machine learning algorithms can be classified into three types.

1.6.1 Supervised learning

Supervised learning is the machine learning task of learning a function that maps an input to an output based on example input-output pairs.

In supervised learning, each example in the training set is a pair consisting of an input object (typically a vector) and an output value. A supervised learning algorithm analyzes the training data and produces a function, which can be used for mapping new examples. In the optimal case, the function will correctly determine the class labels for unseen instances. Both classification and regression problems are supervised learning problems.

A wide range of supervised learning algorithms are available, each with its strengths and weaknesses. There is no single learning algorithm that works best on all supervised learning problems.

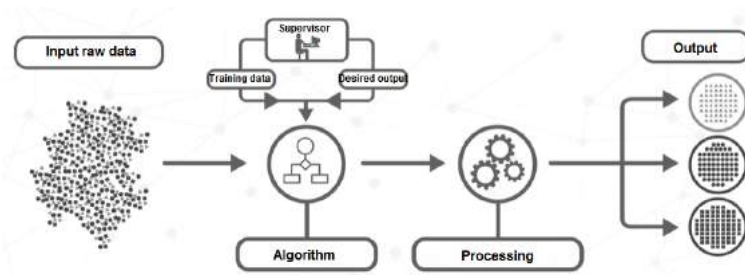


Figure 1.4: Supervised learning

Remarks

A “supervised learning” is so called because the process of an algorithm learning from the training dataset can be thought of as a teacher supervising the learning process. We know the correct answers (that is, the correct outputs), the algorithm iteratively makes predictions on the training data and is corrected by the teacher. Learning stops when the algorithm achieves an acceptable level of performance.

Example

Consider the following data regarding patients entering a clinic. The data consists of the gender and age of the patients and each patient is labeled as “healthy” or “sick”.

gender	age	label
M	48	sick
M	67	sick
F	53	healthy
M	49	healthy
F	34	sick
M	21	healthy

Based on this data, when a new patient enters the clinic, how can one predict whether he/she is healthy or sick?

1.6.2 Unsupervised learning

Unsupervised learning is a type of machine learning algorithm used to draw inferences from datasets consisting of input data without labeled responses.

In unsupervised learning algorithms, a classification or categorization is not included in the observations. There are no output values and so there is no estimation of functions. Since the examples given to the learner are unlabeled, the accuracy of the structure that is output by the algorithm cannot be evaluated.

The most common unsupervised learning method is cluster analysis, which is used for exploratory data analysis to find hidden patterns or grouping in data.

Example

Consider the following data regarding patients entering a clinic. The data consists of the gender and age of the patients.

gender	age
M	48
M	67
F	53
M	49
F	34
M	21

Based on this data, can we infer anything regarding the patients entering the clinic?

1.6.3 Reinforcement learning

Reinforcement learning is the problem of getting an agent to act in the world so as to maximize its rewards.

A learner (the program) is not told what actions to take as in most forms of machine learning, but instead must discover which actions yield the most reward by trying them. In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situations and, through that, all subsequent rewards.

For example, consider teaching a dog a new trick: we cannot tell it what to do, but we can reward/punish it if it does the right/wrong thing. It has to find out what it did that made it get the reward/punishment. We can use a similar method to train computers to do many tasks, such as playing backgammon or chess, scheduling jobs, and controlling robot limbs.

Reinforcement learning is different from supervised learning. Supervised learning is learning from examples provided by a knowledgeable expert.

1.7 Sample questions

(a) Short answer questions

1. What is meant by “learning” in the context of machine learning?
2. List out the types of machine learning.
3. Distinguish between classification and regression.
4. What are the differences between supervised and unsupervised learning?
5. What is meant by supervised classification?
6. Explain supervised learning with an example.
7. What do you mean by reinforcement learning?
8. What is an association rule?
9. Explain the concept of Association rule learning. Give the names of two algorithms for generating association rules.
10. What is a classification problem in machine learning. Illustrate with an example.
11. Give three examples of classification problems from real life situations.
12. What is a discriminant in a classification problem?
13. List three machine learning algorithms for solving classification problems.
14. What is a binary classification problem? Explain with an example. Give also an example for a classification problem which is not binary.
15. What is regression problem. What are the different types of regression?

(b) Long answer questions

1. Give a definition of the term “machine learning”. Explain with an example the concept of learning in the context of machine learning.
2. Describe the basic components of the machine learning process.
3. Describe in detail applications of machine learning in any three different knowledge domains.
4. Describe with an example the concept of association rule learning. Explain how it is made use of in real life situations.
5. What is the classification problem in machine learning? Describe three real life situations in different domains where such problems arise.
6. What is meant by a discriminant of a classification problem? Illustrate the idea with examples.
7. Describe in detail with examples the different types of learning like the supervised learning, etc.

Chapter 2

Some general concepts

In this chapter we introduce some general concepts related to one of the simplest examples of supervised learning, namely, the classification problem. We consider mainly binary classification problems. In this context we introduce the concepts of hypothesis, hypothesis space and version space. We conclude the chapter with a brief discussion on how to select hypothesis models and how to evaluate the performance of a model.

2.1 Input representation

The general classification problem is concerned with assigning a class label to an unknown instance from instances of known assignments of labels. In a real world problem, a given situation or an object will have large number of features which may contribute to the assignment of the labels. But in practice, not all these features may be equally relevant or important. Only those which are significant need be considered as inputs for assigning the class labels. These features are referred to as the “input features” for the problem. They are also said to constitute an “*input representation*” for the problem.

Example

Consider the problem of assigning the label “family car” or “not family car” to cars. Let us assume that the features that separate a family car from other cars are the price and engine power. These attributes or features constitute the input representation for the problem. While deciding on this input representation, we are ignoring various other attributes like seating capacity or colour as irrelevant.

2.2 Hypothesis space

In the following discussions we consider only “binary classification” problems; that is, classification problems with only two class labels. The class labels are usually taken as “1” and “0”. The label “1” may indicate “True”, or “Yes”, or “Pass”, or any such label. The label “0” may indicate “False”, or “No” or “Fail”, or any such label. The examples with class labels 1 are called “positive examples” and examples with labels “0” are called “negative examples”.

2.2.1 Definition

1. Hypothesis

In a binary classification problem, a *hypothesis* is a statement or a proposition purporting to explain a given set of facts or observations.

2. Hypothesis space

The *hypothesis space* for a binary classification problem is a set of hypotheses for the problem that might possibly be returned by it.

3. Consistency and satisfying

Let x be an example in a binary classification problem and let $c(x)$ denote the class label assigned to x ($c(x)$ is 1 or 0). Let D be a set of training examples for the problem. Let h be a hypothesis for the problem and $h(x)$ be the class label assigned to x by the hypothesis h .

- (a) We say that the hypothesis h is *consistent* with the set of training examples D if $h(x) = c(x)$ for all $x \in D$.
- (b) We say that an example x *satisfies* the hypothesis h if $h(x) = 1$.

2.2.2 Examples

1. Consider the set of observations of a variable x with the associated class labels given in Table 2.1:

x	27	15	23	20	25	17	12	30	6	10
Class	1	0	1	1	1	0	0	1	0	0

Table 2.1: Sample data to illustrate the concept of hypotheses

Figure 2.1 shows the data plotted on the x -axis.

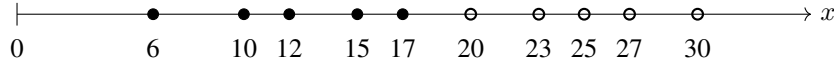


Figure 2.1: Data in Table 2.1 with hollow dots representing positive examples and solid dots representing negative examples

Looking at Figure 2.1, it appears that the class labeling has been done based on the following rule.

$$h' : \text{ IF } x \geq 20 \text{ THEN "1" ELSE "0"}. \quad (2.1)$$

Note that h' is consistent with the training examples in Table 2.1. For example, we have:

$$\begin{aligned} h'(27) &= 1, & c(27) &= 1, & h'(27) &= c(27) \\ h'(15) &= 0, & c(15) &= 0, & h'(15) &= c(15) \end{aligned}$$

Note also that, for $x = 5$ and $x = 28$ (not in training data),

$$h'(5) = 0, \quad h'(28) = 1.$$

The hypothesis h' explains the data. The following proposition also explains the data:

$$h'' : \text{ IF } x \geq 19 \text{ THEN "0" ELSE "1"}. \quad (2.2)$$

It is not enough that the hypothesis explains the given data; it must also predict correctly the class label of future observations. So we consider a set of such hypotheses and choose the "best" one. The set of hypotheses can be defined using a parameter, say m , as given below:

$$h_m : \text{ IF } x \geq m \text{ THEN "1" ELSE "0"}. \quad (2.3)$$

The set of all hypotheses obtained by assigning different values to m constitutes the hypothesis space H ; that is,

$$H = \{h_m : m \text{ is a real number}\}. \quad (2.4)$$

For the same data, we can have different hypothesis spaces. For example, for the data in Table 2.1, we may also consider the hypothesis space defined by the following proposition:

$$h'_m : \text{ IF } x \leq m \text{ THEN "0" ELSE "1"}.$$

2. Consider a situation with four binary variables x_1, x_2, x_3, x_4 and one binary output variable y . Suppose we have the following observations.

x_1	x_2	x_3	x_4	y
0	0	0	1	1
0	1	0	1	0
1	1	0	0	1
0	0	1	0	0

The problem is to predict a function f of x_1, x_2, x_3, x_4 which predicts the value of y for any combination of values of x_1, x_2, x_3, x_4 . In this problem, the hypothesis space is the set of all possible functions f . It can be shown that the size of the hypothesis space is $2^{(2^4)} = 65536$.

3. Consider the problem of assigning the label “family car” or “not family car” to cars. For convenience, we shall replace the label “family car” by “1” and “not family car” by “0”. Suppose we choose the features “price (‘000 \$)” and “power (hp)” as the input representation for the problem. Further, suppose that there is some reason to believe that for a car to be a family car, its price and power should be in certain ranges. This supposition can be formulated in the form of the following proposition:

$$\text{IF } (p_1 < \text{price} < p_2) \text{ AND } (e_1 < \text{power} < e_2) \text{ THEN "1" ELSE "0"} \quad (2.5)$$

for suitable values of p_1, p_2, e_1 and e_2 . Since a solution to the problem is a proposition of the form Eq.(2.5) with specific values for p_1, p_2, e_1 and e_2 , the hypothesis space for the problem is the set of all such propositions obtained by assigning all possible values for p_1, p_2, e_1 and e_2 .

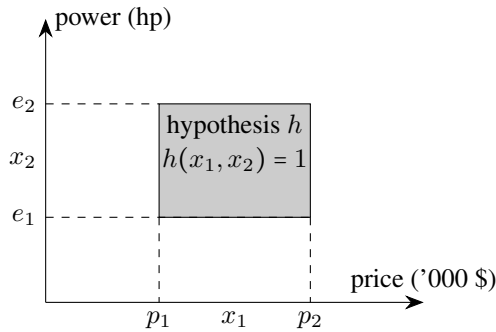


Figure 2.2: An example hypothesis defined by Eq. (2.5)

It is interesting to observe that the set of points in the power–price plane which satisfies the condition

$$(p_1 < \text{price} < p_2) \text{ AND } (e_1 < \text{power} < e_2)$$

defines a rectangular region (minus the boundary) in the price–power space as shown in Figure 2.2. The sides of this rectangular region are parallel to the coordinate axes. Such a rectangle

is called an *axis-aligned rectangle* If h is the hypothesis defined by Eq.(2.5), and (x_1, x_2) is any point in the price–power plane, then $h(x_1, x_2) = 1$ if and only if (x_1, x_2) is within the rectangular region. Hence we may identify the hypothesis h with the rectangular region. Thus, the hypothesis space for the problem can be thought of as the set of all axis-aligned rectangles in the price–power plane.

4. Consider the trading agent trying to infer which books or articles the user reads based on keywords supplied in the article. Suppose the learning agent has the following data (“1” indicates “True” and “0” indicates “False”):

article	crime	academic	local	music	reads
a1	true	false	false	true	1
a2	true	false	false	false	1
a3	false	true	false	false	0
a4	false	false	true	false	0
a5	true	true	false	false	1

The aim is to learn which articles the user reads. The aim is to find a definition such as

IF (crime OR (academic AND (NOT music))) THEN “1” ELSE “0”.

The hypothesis space H could be all boolean combinations of the input features or could be more restricted, such as conjunctions or propositions defined in terms of fewer than three features.

2.3 Ordering of hypotheses

Definition

Let X be the set of all possible examples for a binary classification problem and let h' and h'' be two hypotheses for the problem.

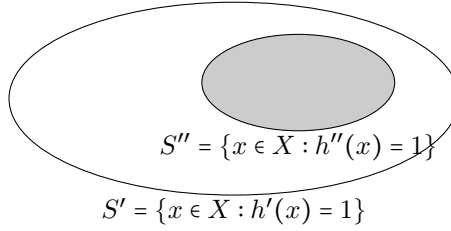


Figure 2.3: Hypothesis h' is more general than hypothesis h'' if and only if $S'' \subseteq S'$

1. We say that h' is *more general than* h'' if and only if for every $x \in X$, if x satisfies h'' then x satisfies h' also; that is, if $h''(x) = 1$ then $h'(x) = 1$ also. The relation “is more general than” defines a partial ordering relation in hypothesis space.
2. We say that h' is *more specific than* h'' , if h'' is more general than h' .
3. We say that h' is *strictly more general than* h'' if h' is more general than h'' and h'' is *not* more general than h' .
4. We say that h' is *strictly more specific than* h'' if h' is more specific than h'' and h'' is *not* more specific than h' .

Example

Consider the hypotheses h' and h'' defined in Eqs.(2.1),(2.2). Then it is easy to check that if $h'(x) = 1$ then $h''(x) = 1$ also. So, h'' is more general than h' . But, h' is not more general than h'' and so h'' is strictly more general than h' .

2.4 Version space**Definition**

Consider a binary classification problem. Let D be a set of training examples and H a hypothesis space for the problem. The *version space* for the problem with respect to the set D and the space H is the set of hypotheses from H consistent with D ; that is, it is the set

$$VS_{D,H} = \{h \in H : h(x) = c(x) \text{ for all } x \in D\}.$$

2.4.1 Examples**Example 1**

Consider the data D given in Table 2.1 and the hypothesis space defined by Eqs.(2.3)-(2.4).

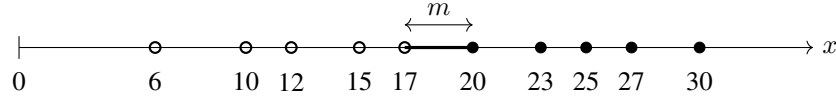


Figure 2.4: Values of m which define the version space with data in Table 2.1 and hypothesis space defined by Eq.(2.4)

From Figure 2.4 we can easily see that the hypothesis space with respect this dataset D and hypothesis space H is as given below:

$$VS_{D,H} = \{h_m : 17 < m \leq 20\}.$$

Example 2

Consider the problem of assigning the label “family car” (indicated by “1”) or “not family car” (indicated by “0”) to cars. Given the following examples for the problem and assuming that the hypothesis space is as defined by Eq. (2.5), the version space for the problem.

x_1 : Price in '000 (\$)	32	82	44	34	43	80	38
x_2 : Power (hp)	170	333	220	235	245	315	215
Class	0	0	1	1	1	0	1

x_1	47	27	56	28	20	25	66	75
x_2	260	290	320	305	160	300	250	340
Class	1	0	0	0	0	0	0	0

Solution

Figure 2.5 shows a scatter plot of the given data. In the figure, the data with class label “1” (family car) is shown as hollow circles and the data with class labels “0” (not family car) are shown as solid dots.

A hypothesis as given by Eq.(2.5) with specific values for the parameters p_1 , p_2 , e_1 and e_2 specifies an axis-aligned rectangle as shown in Figure 2.2. So the hypothesis space for the problem can be thought as the set of axis-aligned rectangles in the price-power plane.

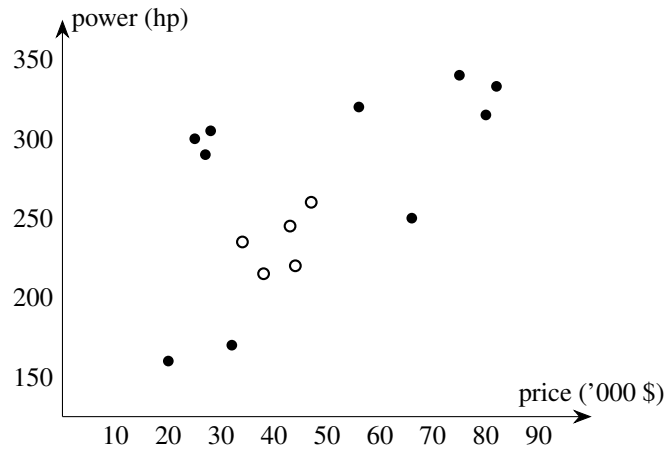


Figure 2.5: Scatter plot of price-power data (hollow circles indicate positive examples and solid dots indicate negative examples)

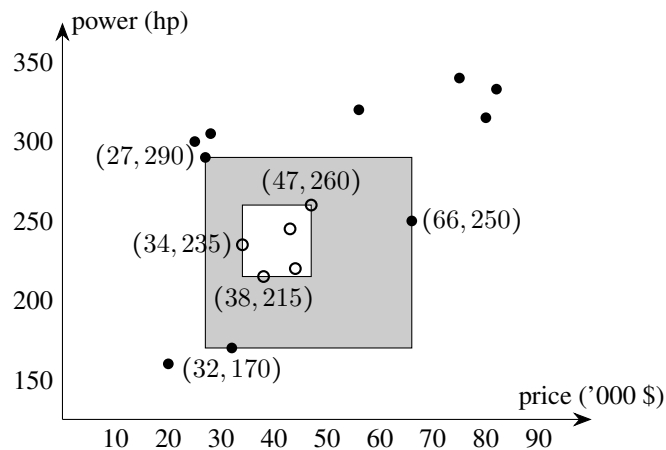


Figure 2.6: The version space consists of hypotheses corresponding to axis-aligned rectangles contained in the shaded region

The version space consists of all hypotheses specified by axis-aligned rectangles contained in the shaded region in Figure 2.6. The inner rectangle is defined by

$$(34 < \text{price} < 47) \text{ AND } (215 < \text{power} < 260)$$

and the outer rectangle is defined by

$$(27 < \text{price} < 66) \text{ AND } (170 < \text{power} < 290).$$

Example 3

Consider the problem of finding a rule for determining days on which one can enjoy water sport. The rule is to depend on a few attributes like “temp”, “humidity”, etc. Suppose we have the following data to help us devise the rule. In the data, a value of “1” for “enjoy” means “yes” and a value of “0” indicates “no”.

Example	sky	temp	humidity	wind	water	forecast	enjoy
1	sunny	warm	normal	strong	warm	same	1
2	sunny	warm	high	strong	warm	same	1
3	rainy	cold	high	strong	warm	change	0
4	sunny	warm	high	strong	cool	change	1

Find the hypothesis space and the version space for the problem. (For a detailed discussion of this problem see [4] Chapter2.)

Solution

We are required to find a rule of the following form, consistent with the data, as a solution of the problem.

$$\begin{aligned}
 &(\text{sky} = x_1) \wedge (\text{temp} = x_2) \wedge (\text{humidity} = x_3) \wedge \\
 &(\text{wind} = x_4) \wedge (\text{water} = x_5) \wedge (\text{forecast} = x_6) \leftrightarrow \text{yes}
 \end{aligned} \tag{2.6}$$

where

$$x_1 = \text{sunny, warm, } *$$

$$x_2 = \text{warm, cold, } *$$

$$x_3 = \text{normal, high, } *$$

$$x_4 = \text{strong, } *$$

$$x_5 = \text{warm, cool, } *$$

$$x_6 = \text{same, change, } *$$

(Here a “*” indicates other possible values of the attributes.) The hypothesis may be represented compactly as a vector

$$(a_1, a_2, a_3, a_4, a_5, a_6)$$

where, in the positions of $a_1, \dots, a_6$, we write

- a “?” to indicate that any value is acceptable for the corresponding attribute,
- a “∅” to indicate that no value is acceptable for the corresponding attribute,
- some specific single required value for the corresponding attribute

For example, the vector

$$(\text{?, cold, high, ?, ?, ?})$$

indicates the hypothesis that one enjoys the sport only if “temp” is “cold” and “humidity” is “high” whatever be the values of the other attributes.

It can be shown that the version space for the problem consists of the following six hypotheses only:

$$\begin{aligned}
 &(\text{sunny, warm, ?, strong, ?, ?}) \\
 &(\text{sunny, ?, ?, strong, ?, ?}) \\
 &(\text{sunny, warm, ?, ?, ?, ?}) \\
 &(\text{?, warm, ?, strong, ?, ?}) \\
 &(\text{sunny, ?, ?, ?, ?, ?}) \\
 &(\text{?, warm, ?, ?, ?, ?})
 \end{aligned}$$

2.5 Noise

2.5.1 Noise and their sources

Noise is any unwanted anomaly in the data ([2] p.25). Noise may arise due to several factors:

1. There may be imprecision in recording the input attributes, which may shift the data points in the input space.
2. There may be errors in labeling the data points, which may relabel positive instances as negative and vice versa. This is sometimes called teacher noise.
3. There may be additional attributes, which we have not taken into account, that affect the label of an instance. Such attributes may be hidden or latent in that they may be unobservable. The effect of these neglected attributes is thus modeled as a random component and is included in “noise.”

2.5.2 Effect of noise

Noise distorts data. When there is noise in data, learning problems may not produce accurate results. Also, simple hypotheses may not be sufficient to explain the data and so complicated hypotheses may have to be formulated. This leads to the use of additional computing resources and the needless wastage of such resources.

For example, in a binary classification problem with two variables, when there is noise, there may not be a simple boundary between the positive and negative instances and to separate them. A rectangle can be defined by four numbers, but to define a more complicated shape one needs a more complex model with a much larger number of parameters. So, when there is noise, we may make a complex model which makes a perfect fit to the data and attain zero error; or, we may use a simple model and allow some error.

2.6 Learning multiple classes

So far we have been discussing binary classification problems. In a general case there may be more than two classes. Two methods are generally used to handle such cases. These methods are known by the names “one-against-all” and “one-against-one”.

2.6.1 Procedures for learning multiple classes

“One-against all” method

Consider the case where there are K classes denoted by $C_1, \dots, C_K$. Each input instance belongs to exactly one of them.

We view a K -class classification problem as K two-class problems. In the i -th two-class problem, the training examples belonging to C_i are taken as the positive examples and the examples of all other classes are taken as the negative examples. So, we have to find K hypotheses $h_1, \dots, h_K$ where h_i is defined by

$$h_i(x) = \begin{cases} 1 & \text{if } x \text{ is in class } C_i \\ 0 & \text{otherwise} \end{cases}$$

For a given x , ideally only one of $h_i(x)$ is 1 and then we assign the class C_i to x . But, when no, or, two or more, $h_i(x)$ is 1, we cannot choose a class. In such a case, we say that the classifier *rejects* such cases.

“One-against-one” method

In the *one-against-one* (OAO) (also called *one-vs-one* (OVO)) strategy, a classifier is constructed for each pair of classes. If there are K different class labels, a total of $K(K - 1)/2$ classifiers are constructed. An unknown instance is classified with the class getting the most votes. Ties are broken arbitrarily.

For example, let there be three classes, A , B and C . In the OVO method we construct $3(3 - 1)/2 = 3$ binary classifiers. Now, if any x is to be classified, we apply each of the three classifiers to x . Let the three classifiers assign the classes A , B , B respectively to x . Since a label to x is assigned by the majority voting, in this example, we assign the class label of B to x .

2.7 Model selection

As we have pointed earlier in Section 1.1.1, there is no universally accepted definition of the term “model”. It may be understood as some mathematical expression or equation, or some mathematical structures such as graphs and trees, or a division of sets into disjoint subsets, or a set of logical “if ... then ... else ...” rules, or some such thing.

In order to formulate a hypothesis for a problem, we have to choose some model and the term “model selection” has been used to refer to the process of choosing a model. However, the term has been used to indicate several things. In some contexts it has been used to indicate the process of choosing one particular approach from among several different approaches. This may be choosing an appropriate algorithms from a selection of possible algorithms, or choosing the sets of features to be used for input, or choosing initial values for certain parameters. Sometimes “model selection” refers to the process of picking a particular mathematical model from among different mathematical models which all purport to describe the same data set. It has also been described as the process of choosing the right inductive bias.

2.7.1 Inductive bias

In a learning problem we only have the data. But data by itself is not sufficient to find the solution. We should make some extra assumptions to have a solution with the data we have. The set of assumptions we make to have learning possible is called the *inductive bias* of the learning algorithm. One way we introduce inductive bias is when we assume a hypothesis class.

Examples

- In learning the class of family car, there are infinitely many ways of separating the positive examples from the negative examples. Assuming the shape of a rectangle is an inductive bias.
- In regression, assuming a linear function is an inductive bias.

The model selection is about choosing the right inductive bias.

2.7.2 Advantages of a simple model

Even though a complex model may not be making any errors in prediction, there are certain advantages in using a simple model.

1. A simple model is easy to use.
2. A simple model is easy to train. It is likely to have fewer parameters.
It is easier to find the corner values of a rectangle than the control points of an arbitrary shape.
3. A simple model is easy to explain.

4. A simple model would generalize better than a complex model. This principle is known as Occam's razor, which states that simpler explanations are more plausible and any unnecessary complexity should be shaved off.

Remarks

A model should not be too simple! With a small training set when the training instances differ a little bit, we expect the simpler model to change less than a complex model: A simple model is thus said to have less *variance*. On the other hand, a too simple model assumes more, is more rigid, and may fail if indeed the underlying class is not that simple. A simpler model has more bias. Finding the optimal model corresponds to minimizing both the bias and the variance.

2.8 Generalisation

How well a model trained on the training set predicts the right output for new instances is called *generalization*.

Generalization refers to how well the concepts learned by a machine learning model apply to specific examples not seen by the model when it was learning. The goal of a good machine learning model is to generalize well from the training data to any data from the problem domain. This allows us to make predictions in the future on data the model has never seen. Overfitting and underfitting are the two biggest causes for poor performance of machine learning algorithms. The model should be selected having the best generalisation. This is said to be the case if these problems are avoided.

- **Underfitting**

Underfitting is the production of a machine learning model that is not complex enough to accurately capture relationships between a dataset's features and a target variable.

- **Overfitting**

Overfitting is the production of an analysis which corresponds too closely or exactly to a particular set of data, and may therefore fail to fit additional data or predict future observations reliably.

Example 1

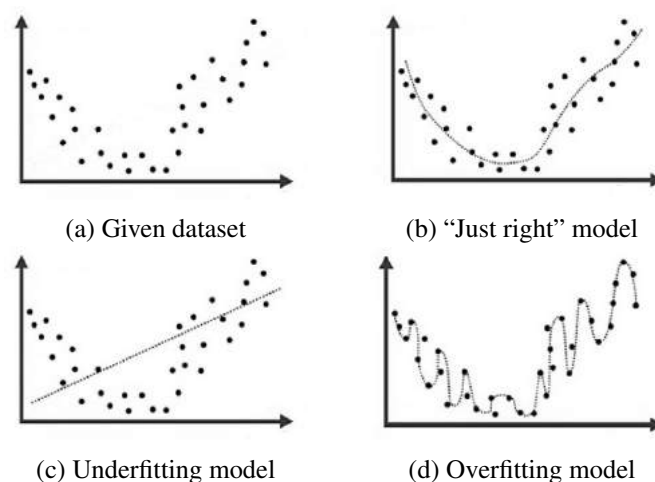


Figure 2.7: Examples for overfitting and overfitting models

Consider a dataset shown in Figure 2.7(a). Let it be required to fit a regression model to the data. The graph of a model which looks “just right” is shown in Figure 2.7(b). In Figure 2.7(c) we have a linear regression model for the same dataset and this model does seem to capture the essential features of the dataset. So this model suffers from underfitting. In Figure 2.7(d) we have a regression model which corresponds too closely to the given dataset and hence it does not account for small random noises in the dataset. Hence it suffers from overfitting.

Example 2

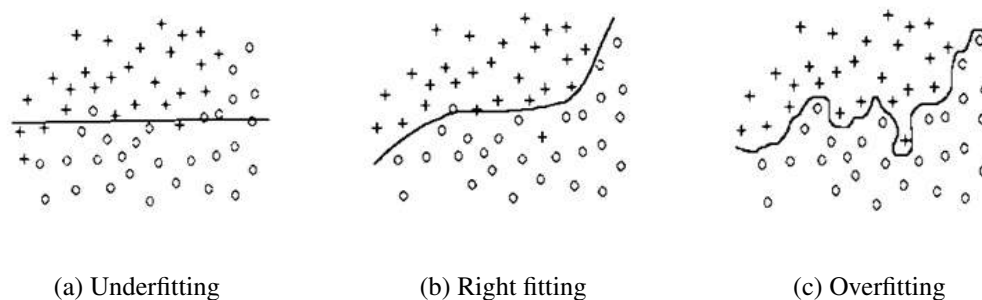


Figure 2.8: Fitting a classification boundary

Suppose we have to determine the classification boundary for a dataset two class labels. An example situation is shown in Figure 2.8 where the curved line is the classification boundary. The three figures illustrate the cases of underfitting, right fitting and overfitting.

2.8.1 Testing generalisation: Cross-validation

We can measure the generalization ability of a hypothesis, namely, the quality of its inductive bias, if we have access to data outside the training set. We simulate this by dividing the training set we have into two parts. We use one part for training (that is, to find a hypothesis), and the remaining part is called the *validation set* and is used to test the generalization ability. Assuming large enough training and validation sets, the hypothesis that is the most accurate on the validation set is the best one (the one that has the best inductive bias). This process is called *cross-validation*.

2.9 Sample questions

(a) Short answer questions

1. Explain the general-to-specific ordering of hypotheses.
2. In the context of classification problems explain with examples the following: (i) hypothesis (ii) hypothesis space.
3. Define the version space of a binary classification problem.
4. Explain the “one-against-all” method for learning multiple classes.
5. Describe the “one-against-one” method for learning multiple classes.
6. What is meant by inductive bias in machine learning? Give an example.
7. What is meant by overfitting of data? Explain with an example.
8. What is meant by overfitting and underfitting of data with examples.

(b) Long answer questions

1. Define version space and illustrate it with an example.
2. Given the following data

x	0	3	5	9	12	18	23
Label	0	0	0	1	1	1	1

and the hypothesis space

$$H = \{h_m \mid m \text{ a real number}\}$$

where h_m is defined by

$$\text{IF } x \leq m \text{ THEN } 1 \text{ ELSE } 0,$$

find the version space the problem with respect to D and H .

3. What is meant by “noise” in data? What are its sources and how it is affecting results?
4. Consider the following data:

x	2	3	5	8	10	15	16	18	20
y	12	15	10	6	8	10	7	9	10
Class label	0	0	1	1	1	1	0	0	0

Determine the version space if the hypothesis space consists of all hypotheses of the form

$$\text{IF } (x_1 < x < x_2) \text{ AND } (y_1 < y < y_2) \text{ THEN "1" ELSE "0"}.$$

5. For the data in problem 4, what would be the version space if the hypothesis space consists of all hypotheses of the form

$$\text{IF } (x - x_1)^2 + (y - y_1)^2 \leq r^2 \text{ THEN "1" ELSE "0"}.$$

6. What issues are to be considered while selecting a model for applying machine learning in a given problem.

Chapter 3

VC dimension and PAC learning

The concepts of Vapnik-Chervonenkis dimension (VC dimension) and probably approximate correct (PAC) learning are two important concepts in the mathematical theory of learnability and hence are mathematically oriented. The former is a measure of the capacity (complexity, expressive power, richness, or flexibility) of a space of functions that can be learned by a classification algorithm. It was originally defined by Vladimir Vapnik and Alexey Chervonenkis in 1971. The latter is a framework for the mathematical analysis of learning algorithms. The goal is to check whether the probability for a selected hypothesis to be approximately correct is very high. The notion of PAC learning was proposed by Leslie Valiant in 1984.

3.1 Vapnik-Chervonenkis dimension

Let H be the hypothesis space for some machine learning problem. The Vapnik-Chervonenkis dimension of H , also called the VC dimension of H , and denoted by $VC(H)$, is a measure of the complexity (or, capacity, expressive power, richness, or flexibility) of the space H . To define the VC dimension we require the notion of the shattering of a set of instances.

3.1.1 Shattering of a set

Let D be a dataset containing N examples for a binary classification problem with class labels 0 and 1. Let H be a hypothesis space for the problem. Each hypothesis h in H partitions D into two disjoint subsets as follows:

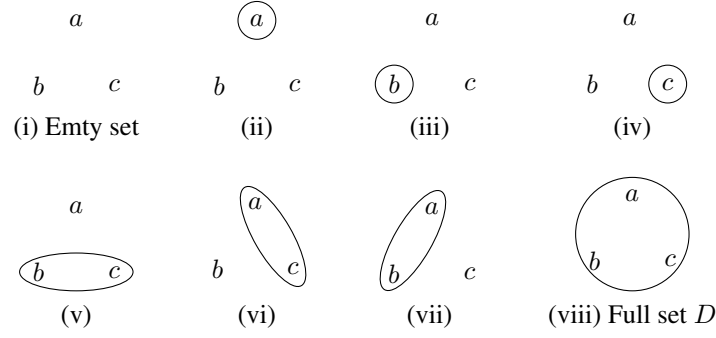
$$\{x \in D \mid h(x) = 0\} \text{ and } \{x \in D \mid h(x) = 1\}.$$

Such a partition of S is called a “*dichotomy*” in D . It can be shown that there are 2^N possible dichotomies in D . To each dichotomy of D there is a unique assignment of the labels “1” and “0” to the elements of D . Conversely, if S is any subset of D then, S defines a unique hypothesis h as follows:

$$h(x) = \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Thus to specify a hypothesis h , we need only specify the set $\{x \in D \mid h(x) = 1\}$.

Figure 3.1 shows all possible dichotomies of D if D has three elements. In the figure, we have shown only one of the two sets in a dichotomy, namely the set $\{x \in D \mid h(x) = 1\}$. The circles and ellipses represent such sets.

Figure 3.1: Different forms of the set $\{x \in S : h(x) = 1\}$ for $D = \{a, b, c\}$

We require the notion of a hypothesis consistent with a set of examples introduced in Section 2.4 in the following definition.

Definition

A set of examples D is said to be *shattered* by a hypothesis space H if and only if for every dichotomy of D there exists some hypothesis in H consistent with the dichotomy of D .

3.1.2 Vapnik-Chervonenkis dimension

The following example illustrates the concept of Vapnik-Chervonenkis dimension.

Example

Let the instance space X be the set of all real numbers. Consider the hypothesis space defined by Eqs.(2.3)-(2.4):

$$H = \{h_m : m \text{ is a real number}\},$$

where

$$h_m : \text{ IF } x \geq m \text{ THEN "1" ELSE "0"}.$$

- i) Let D be a subset of X containing only a single number, say, $D = \{3.5\}$. There are 2 dichotomies for this set. These correspond to the following assignment of class labels:

x	3.25	x	3.25
Label	0	Label	1

$h_4 \in H$ is consistent with the former dichotomy and $h_3 \in H$ is consistent with the latter. So, to every dichotomy in D there is a hypothesis in H consistent with the dichotomy. Therefore, the set D is shattered by the hypothesis space H .

- ii) Let D be a subset of X containing two elements, say, $D = \{3.25, 4.75\}$. There are 4 dichotomies in D and they correspond to the assignment of class labels shown in Table 3.1.

In these dichotomies, h_5 is consistent with (a), h_4 is consistent with (b) and h_3 is consistent with (d). But there is no hypothesis $h_m \in H$ consistent with (c). Thus the two-element set D is not shattered by H . In a similar way it can be shown that there is no two-element subset of X which is shattered by H .

It follows that the size of the largest finite subset of X shattered by H is 1. This number is the VC dimension of H .

x	3.25	4.75
Label	0	0
(a)		

x	3.25	4.75
Label	0	1
(b)		

x	3.25	4.75
Label	1	0
(c)		

x	3.25	4.75
Label	1	1
(d)		

Table 3.1: Different assignments of class labels to the elements of $\{3.25, 4.75\}$ **Definition**

The *Vapnik-Chervonenkis dimension* (*VC dimension*) of a hypothesis space H defined over an instance space (that is, the set of all possible examples) X , denoted by $VC(H)$, is the size of the largest finite subset of X shattered by H . If arbitrarily large subsets of X can be shattered by H , then we define $VC(H) = \infty$.

Remarks

It can be shown that $VC(H) \leq \log_2(|H|)$ where H is the number of hypotheses in H .

3.1.3 Examples

1. Let X be the set of all real numbers (say, for example, the set of heights of people). For any real numbers a and b define a hypothesis $h_{a,b}$ as follows:

$$h_{a,b}(x) = \begin{cases} 1 & \text{if } a < x < b \\ 0 & \text{otherwise} \end{cases}$$

Let the hypothesis space H consist of all hypotheses of the form $h_{a,b}$. We show that $VC(H) = 2$.

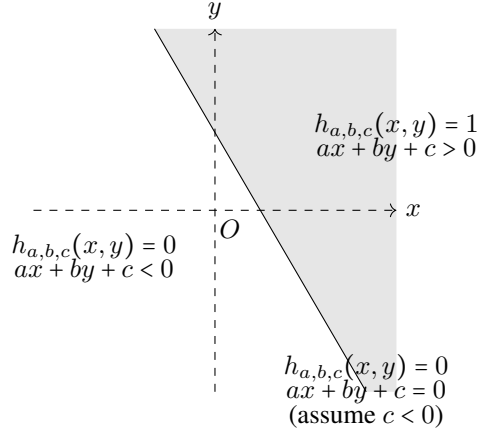
2. We have to show that there is a subset of X of size 2 shattered by H and there is no subset of size 3 shattered by H .

- Consider the two-element set $D = \{3.25, 4.75\}$. The various dichotomies of D are given in Table 3.1. It can be seen that the hypothesis $h_{5,6}$ is consistent with (a), $h_{4,5}$ is consistent with (b), $h_{3,4}$ is consistent with (c) and $h_{3,5}$ is consistent with (d). So the set D is shattered by H .
- Consider a three-element subset $D = \{x_1, x_2, x_3\}$. Let us assume that $x_1 < x_2 < x_3$. H cannot shatter this subset because the dichotomy represented by the set $\{x_1, x_3\}$ cannot be represented by a hypothesis in H (any interval containing both x_1 and x_3 will contain x_2 also).

Therefore, the size of the largest subset of X shattered by H is 2 and so $VC(H) = 2$.

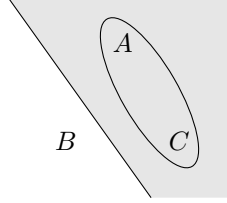
2. Let the instance space X be the set of all points (x, y) in a plane. For any three real numbers, a, b, c define a class labeling as follows:

$$h_{a,b,c}(x, y) = \begin{cases} 1 & \text{if } ax + by + c > 0 \\ 0 & \text{otherwise} \end{cases}$$

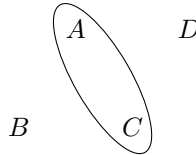
Figure 3.2: Geometrical representation of the hypothesis $h_{a,b,c}$

Let H be the set of all hypotheses of the form $h_{a,b,c}$. We show that $VC(H) = 3$. We have show that there is a subset of size 3 shattered by H and there is no subset of size 4 shattered by H .

- Consider a set $D = \{A, B, C\}$ of three non-collinear points in the plane. There are 8 subsets of D and each of these defines a dichotomy of D . We can easily find 8 hypotheses corresponding to the dichotomies defined by these subsets (see Figure 3.3).

Figure 3.3: A hypothesis $h_{a,b,c}$ consistent with the dichotomy defined by the subset $\{A, C\}$ of $\{A, B, C\}$

- Consider a set $S = \{A, B, C, D\}$ of four points in the plane. Let no three of these points be collinear. Then, the points form a quadrilateral. It can be easily seen that, in this case, there is no hypothesis for which the two element set formed by the ends of a diagonal is the corresponding dichotomy (see Figure 3.4).

Figure 3.4: There is no hypothesis $h_{a,b,c}$ consistent with the dichotomy defined by the subset $\{A, C\}$ of $\{A, B, C, D\}$

So the set cannot be shattered by H . If any three of them are collinear, then by some trial and error, it can be seen that in this case also the set cannot be shattered by H . No set with four elements cannot be shattered by H .

From the above discussion we conclude that $VC(H) = 3$.

3. Let X be set of all conjunctions of n boolean literals. Let the hypothesis space H consists of conjunctions of up to n literals. It can be shown that $VC(H) = n$. (The full details of the proof of this is beyond the scope of these notes.)

3.2 Probably approximately correct learning

In computer science, *computational learning theory* (or just *learning theory*) is a subfield of artificial intelligence devoted to studying the design and analysis of machine learning algorithms. In computational learning theory, *probably approximately correct learning* (PAC learning) is a framework for mathematical analysis of machine learning algorithms. It was proposed in 1984 by Leslie Valiant.

In this framework, the learner (that is, the algorithm) receives samples and must select a hypothesis from a certain class of hypotheses. The goal is that, with high probability (the “probably” part), the selected hypothesis will have low generalization error (the “approximately correct” part).

In this section we first give an informal definition of PAC-learnability. After introducing a few more notions, we give a more formal, mathematically oriented, definition of PAC-learnability. At the end, we mention one of the applications of PAC-learnability.

3.2.1 PAC-learnability

To define PAC-learnability we require some specific terminology and related notations.

- Let X be a set called the *instance space* which may be finite or infinite. For example, X may be the set of all points in a plane.
- A *concept class* C for X is a family of functions $c : X \rightarrow \{0, 1\}$. A member of C is called a *concept*. A concept can also be thought of as a subset of X . If C is a subset of X , it defines a unique function $\mu_C : X \rightarrow \{0, 1\}$ as follows:

$$\mu_C(x) = \begin{cases} 1 & \text{if } x \in C \\ 0 & \text{otherwise} \end{cases}$$

- A *hypothesis* h is also a function $h : X \rightarrow \{0, 1\}$. So, as in the case of concepts, a hypothesis can also be thought of as a subset of X . H will denote a set of hypotheses.
- We assume that F is an arbitrary, but fixed, *probability distribution* over X .
- Training examples are obtained by taking random samples from X . We assume that the samples are randomly generated from X according to the probability distribution F .

Now, we give below an informal definition of PAC-learnability.

Definition (informal)

Let X be an instance space, C a concept class for X , h a hypothesis in C and F an arbitrary, but fixed, probability distribution. The concept class C is said to be *PAC-learnable* if there is an algorithm A which, for samples drawn with any probability distribution F and any concept $c \in C$, will with high probability produce a hypothesis $h \in C$ whose error is small.

Additional notions

• True error

To formally define PAC-learnability, we require the concept of the true error of a hypothesis h with respect to a target concept c denoted by $\text{error}_F(h)$. It is defined by

$$\text{error}_F(h) = P_{x \in F}(h(x) \neq c(x))$$

where the notation $P_{x \in F}$ indicates that the probability is taken for x drawn from X according to the distribution F . This error is the probability that h will misclassify an instance x drawn at random from X according to the distribution F . This error is not directly observable to the learner; it can only see the training error of each hypothesis (that is, how often $h(x) \neq c(x)$ over training instances).

- **Length or dimension of an instance**

We require the notion of the length or dimension or size of an instance in the instance space X . If the instance space X is the n -dimensional Euclidean space, then each example is specified by n real numbers and so the length of the examples may be taken as n . Similarly, if X is the space of the conjunctions of n Boolean literals, then the length of the examples may be taken as n . These are the commonly considered instance spaces in computational learning theory.

- **Size of a concept**

We need the notion of the size of a concept c . For any concept c , we define $\text{size}(c)$ to be the size of the smallest representation of c using some finite alphabet Σ .

(For a detailed discussion of these and related ideas, see [6] pp.7-15.)

Definition ([4] p.206)

Consider a concept class C defined over a set of instances X of length n and a learner (algorithm) L using hypothesis space H . C is said to be PAC-learnable by L using H if for all $c \in C$, distribution F over X , ϵ such that $0 < \epsilon < 1/2$ and δ such that $0 < \delta < 1/2$, learner L will with probability at least $(1 - \delta)$ output a hypothesis h such that $\text{error}_F(h) \leq \epsilon$, in time that is polynomial in $1/\epsilon$, $1/\delta$, n and $\text{size}(c)$.

3.2.2 Examples

To illustrate the definition of PAC-learnability, let us consider some concrete examples.

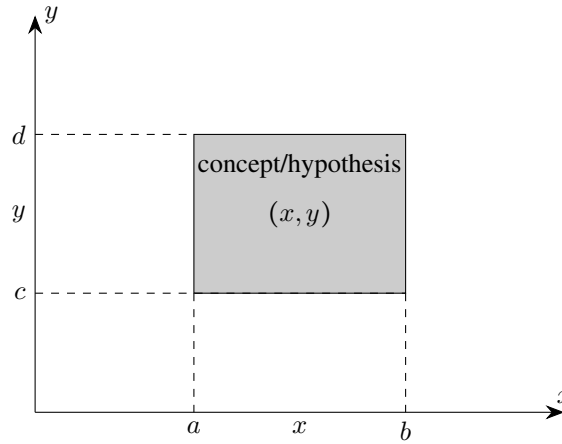


Figure 3.5: An axis-aligned rectangle in the Euclidean plane

Example 1

- Let the instance space be the set X of all points in the Euclidean plane. Each point is represented by its coordinates (x, y) . So, the dimension or length of the instances is 2.
- Let the concept class C be the set of all “axis-aligned rectangles” in the plane; that is, the set of all rectangles whose sides are parallel to the coordinate axes in the plane (see Figure 3.5).
- Since an axis-aligned rectangle can be defined by a set of inequalities of the following form having four parameters

$$a \leq x \leq b, \quad c \leq y \leq d$$

the size of a concept is 4.

- We take the set H of all hypotheses to be equal to the set C of concepts, $H = C$.

- Given a set of sample points labeled positive or negative, let L be the algorithm which outputs the hypothesis defined by the axis-aligned rectangle which gives the tightest fit to the positive examples (that is, that rectangle with the smallest area that includes all of the positive examples and none of the negative examples) (see Figure 3.6).

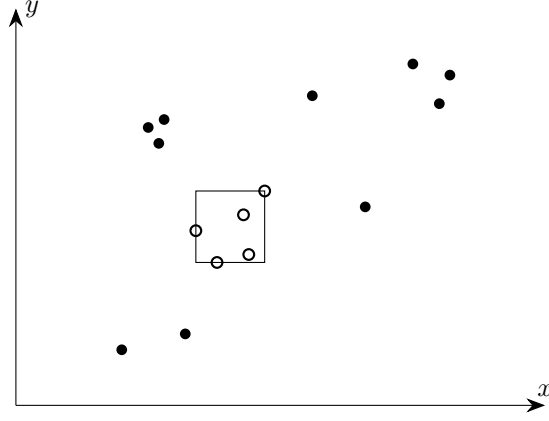


Figure 3.6: Axis-aligned rectangle which gives the tightest fit to the positive examples

It can be shown that, in the notations introduced above, the concept class C is PAC-learnable by the algorithm L using the hypothesis space H of all axis-aligned rectangles.

Example 2

- Let X the set of all n -bit strings. Each n -bit string may be represented by an ordered n -tuple $(a_1, \dots, a_n)$ where each a_i is either 0 or 1. This may be thought of as an assignment of 0 or 1 to n boolean variables $x_1, \dots, x_n$. The set X is sometimes denoted by $\{0, 1\}^n$.
- To define the concept class, we distinguish certain subsets of X in a special way. By a literal we mean, a Boolean variable x_i or its negation $\neg x_i$. We consider conjunctions of literals over $x_1, \dots, x_n$. Each conjunction defines a subset of X . for example, the conjunction $x_1 \wedge \neg x_2 \wedge x_4$ defines the following subset of X :

$$\{a = (a_1, \dots, a_n) \in X \mid a_1 = 1, a_2 = 0, a_4 = 1\}$$

The concept class C consists of all subsets of X defined by conjunctions of Boolean literals over $x_1, \dots, x_n$.

- The hypothesis class H is defined as equal to the concept class C .
- Let L be a certain algorithm called “Find-S algorithm” used to find a most specific hypothesis (see [4] p.26).

The concept class C of all subsets of $X = \{0, 1\}^n$ defined by conjunctions of Boolean literals over $x_1, \dots, x_n$ is PAC-learnable by the Find-S algorithm using the hypothesis space $H = C$.

3.2.3 Applications

To make the discussions complete, we introduce one simple application of the PAC-learning theory. The application is the derivation of a mathematical expression to estimate the size of samples that would produce a hypothesis with a given high probability and which has a generalization error of given low probability.

We use the following assumptions and notations:

- We assume that the hypothesis space H is *finite*. Let $|H|$ denote the number of elements in H .

- We assume that the concept class C be equal to H .
- Let m be the number of elements in the set of samples.
- Let ϵ and δ be such that $0 < \epsilon, \delta < 1$.
- The algorithm can be any *consistent algorithm*, that is, any algorithm which correctly classifies the training examples.

It can be shown that, if m is chosen such that

$$m \geq \frac{1}{\epsilon} (\ln(|H|) + \ln(1/\delta))$$

then any consistent algorithm will successfully produce any concept in H with probability $(1 - \delta)$ and with an error having a maximum probability of ϵ .

3.3 Sample questions

(a) Short answer questions

1. What is VC dimension?
2. Explain Vapnik-Chervonenkis dimension.
3. Give an informal definition of PAC learnability.
4. Give a precise definition of PAC learnability.
5. Give an application of PAC learnable algorithm.

(b) Long answer questions

1. Let X be the set of all real numbers. Describe a hypothesis for X for which the VC dimension is 0.
2. Let X be the set of all real numbers. Describe a hypothesis for X for which the VC dimension is 1.
3. Let X be the set of all real numbers. Describe a hypothesis for X for which the VC dimension is 2. Describe an example for which the VC dimension is 3.
4. Describe an example of a PAC learnable concept class.
5. An open interval in $\mathbb{R}$ is defined as $(a, b) = \{x \in \mathbb{R} \mid a < x < b\}$. It has two parameters a and b . Show that the sets of all open intervals has a VC dimension of 2.

Chapter 4

Dimensionality reduction

The complexity of any classifier or regressor depends on the number of inputs. This determines both the time and space complexity and the necessary number of training examples to train such a classifier or regressor. In this chapter, we discuss various methods for decreasing input dimensionality without losing accuracy.

4.1 Introduction

In many learning problems, the datasets have large number of variables. Sometimes, the number of variables is more than the number of observations. For example, such situations have arisen in many scientific fields such as image processing, mass spectrometry, time series analysis, internet search engines, and automatic text analysis among others. Statistical and machine learning methods have some difficulty when dealing with such high-dimensional data. Normally the number of input variables is reduced before the machine learning algorithms can be successfully applied.

In statistical and machine learning, dimensionality reduction or dimension reduction is the process of reducing the number of variables under consideration by obtaining a smaller set of principal variables.

Dimensionality reduction may be implemented in two ways.

- **Feature selection**

In feature selection, we are interested in finding k of the total of n features that give us the most information and we discard the other $(n - k)$ dimensions. We are going to discuss *subset selection* as a feature selection method.

- **Feature extraction**

In feature extraction, we are interested in finding a new set of k features that are the combination of the original n features. These methods may be supervised or unsupervised depending on whether or not they use the output information. The best known and most widely used feature extraction methods are *Principal Components Analysis* (PCA) and *Linear Discriminant Analysis* (LDA), which are both linear projection methods, unsupervised and supervised respectively.

Measures of error

In both methods we require a measure of the error in the model.

- In regression problems, we may use the *Mean Squared Error* (MSE) or the *Root Mean Squared Error* (RMSE) as the measure of error. MSE is the sum, over all the data points, of the square of the difference between the predicted and actual target variables, divided by

the number of data points. If $y_1, \dots, y_n$ are the observed values and $\hat{y}_1, \dots, \hat{y}_n$ are the predicted values, then

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- In classification problems, we may use the *misclassification rate* as a measure of the error. This is defined as follows:

$$\text{misclassification rate} = \frac{\text{no. of misclassified examples}}{\text{total no. of examples}}$$

4.2 Why dimensionality reduction is useful

There are several reasons why we are interested in reducing dimensionality.

- In most learning algorithms, the complexity depends on the number of input dimensions, d , as well as on the size of the data sample, N , and for reduced memory and computation, we are interested in reducing the dimensionality of the problem. Decreasing d also decreases the complexity of the inference algorithm during testing.
- When an input is decided to be unnecessary, we save the cost of extracting it.
- Simpler models are more robust on small datasets. Simpler models have less variance, that is, they vary less depending on the particulars of a sample, including noise, outliers, and so forth.
- When data can be explained with fewer features, we get a better idea about the process that underlies the data, which allows knowledge extraction.
- When data can be represented in a few dimensions without loss of information, it can be plotted and analyzed visually for structure and outliers.

4.3 Subset selection

In machine learning *subset selection*, sometimes also called *feature selection*, or *variable selection*, or *attribute selection*, is the process of selecting a subset of relevant features (variables, predictors) for use in model construction.

Feature selection techniques are used for four reasons:

- simplification of models to make them easier to interpret by researchers/users
- shorter training times,
- to avoid the curse of dimensionality
- enhanced generalization by reducing overfitting

The central premise when using a feature selection technique is that the data contains many features that are either redundant or irrelevant, and can thus be removed without incurring much loss of information.

There are several approaches to subset selection. In these notes, we discuss two of the simplest approaches known as forward selection and backward selection methods.

4.3.1 Forward selection

In *forward selection*, we start with no variables and add them one by one, at each step adding the one that decreases the error the most, until any further addition does not decrease the error (or decreases it only slightly).

Procedure

We use the following notations:

- n : number of input variables
- $x_1, \dots, x_n$: input variables
- F_i : a subset of the set of input variables
- $E(F_i)$: error incurred on the validation sample when only the inputs in F_i are used

1. Set $F_0 = \emptyset$ and $E(F_0) = \infty$.
2. For $i = 0, 1, \dots$, repeat the following until $E(F_{i+1}) \geq E(F_i)$:
 - (a) For all possible input variables x_j , train the model with the input variables $F_i \cup \{x_j\}$ and calculate $E(F_i \cup \{x_j\})$ on the validation set.
 - (b) Choose that input variable x_m that causes the least error $E(F_i \cup \{x_j\})$:

$$m = \arg \min_j E(F_i \cup \{x_j\})$$
 - (c) Set $F_{i+1} = F_i \cup \{x_m\}$.
3. The set F_i is outputted as the best subset.

Remarks

1. In this procedure, we stop if adding any feature does not decrease the error E . We may even decide to stop earlier if the decrease in error is too small, where there is a user-defined threshold that depends on the application constraints.
2. This process may be costly because to decrease the dimensions from n to k , we need to train and test the system

$$n + (n-1) + (n-2) + \dots + (n-k)$$

times, which is $O(n^2)$.

4.3.2 Backward selection

In sequential backward selection, we start with the set containing all features and at each step remove the one feature that causes the least error.

Procedure

We use the following notations:

- n : number of input variables
- $x_1, \dots, x_n$: input variables
- F_i : a subset of the set of input variables
- $E(F_i)$: error incurred on the validation sample when only the inputs in F_i are used

1. Set $F_0 = \{x_1, \dots, x_n\}$ and $E(F_0) = \infty$.
2. For $i = 0, 1, \dots$, repeat the following until $E(F_{i+1}) \geq E(F_i)$:
 - (a) For all possible input variables x_j , train the model with the input variables $F_i - \{x_j\}$ and calculate $E(F_i - \{x_j\})$ on the validation set.
 - (b) Choose that input variable x_m that causes the least error $E(F_i - \{x_j\})$:

$$m = \arg \min_j E(F_i - \{x_j\})$$

(c) Set $F_{i+1} = F_i - \{x_m\}$.

3. The set F_i is outputted as the best subset.

4.4 Principal component analysis

Principal component analysis (PCA) is a statistical procedure that uses an orthogonal transformation to convert a set of observations of possibly correlated variables into a set of values of linearly uncorrelated variables called principal components. The number of principal components is less than or equal to the smaller of the number of original variables or the number of observations. This transformation is defined in such a way that the first principal component has the largest possible variance (that is, accounts for as much of the variability in the data as possible), and each succeeding component in turn has the highest variance possible under the constraint that it is orthogonal to the preceding components.

4.4.1 Graphical illustration of the idea

Consider a two-dimensional data, that is, a dataset consisting of examples having two features. Let each of the features be numeric data. So, each example can be plotted on a coordinate plane (x -coordinate indicating the first feature and y -coordinate indicating the second feature). Plotting the example, we get a scatter diagram of the data. Now let us examine some typical scatter diagram and make some observations regarding the directions in which the points in the scatter diagram are spread out.

Let us examine the figures in Figure 4.1.

- (i) Figure 4.1a shows a scatter diagram of a two-dimensional data.
- (ii) Figure 4.1b shows spread of the data in the x direction and Figure 4.1c shows the spread of the data in the y -direction. We note that the spread in the x -direction is more than the spread in the y direction.
- (iii) Examining Figures 4.1d and 4.1e, we note that the maximum spread occurs in the direction shown in Figure 4.1e. Figure 4.1e also shows the point whose coordinates are the mean values of the two features in the dataset. This direction is called the *direction of the first principal component* of the given dataset.
- (iv) The direction which is perpendicular (orthogonal) to the direction of the first principal component is called the *direction of the second principal component* of the dataset. This direction is shown in Figure 4.1f. (This is only with reference to a two-dimensional dataset.)
- (v) The unit vectors along the directions of principal components are called the *principal component vectors*, or simply, *principal components*. These are shown in Figure 4.1g.

Remark

let us consider a dataset consisting of examples with three or more features. In such a case, we have an n -dimensional dataset with $n \geq 3$. In this case, the first principal component is defined exactly as in item iii above. But, for the second component, it may be noted that there would be many directions perpendicular to the direction of the first principal component. The direction of the second principal component is that direction, which is perpendicular to the first principal component, in which the spread of data is largest. The third and higher order principal components are constructed in a similar way.

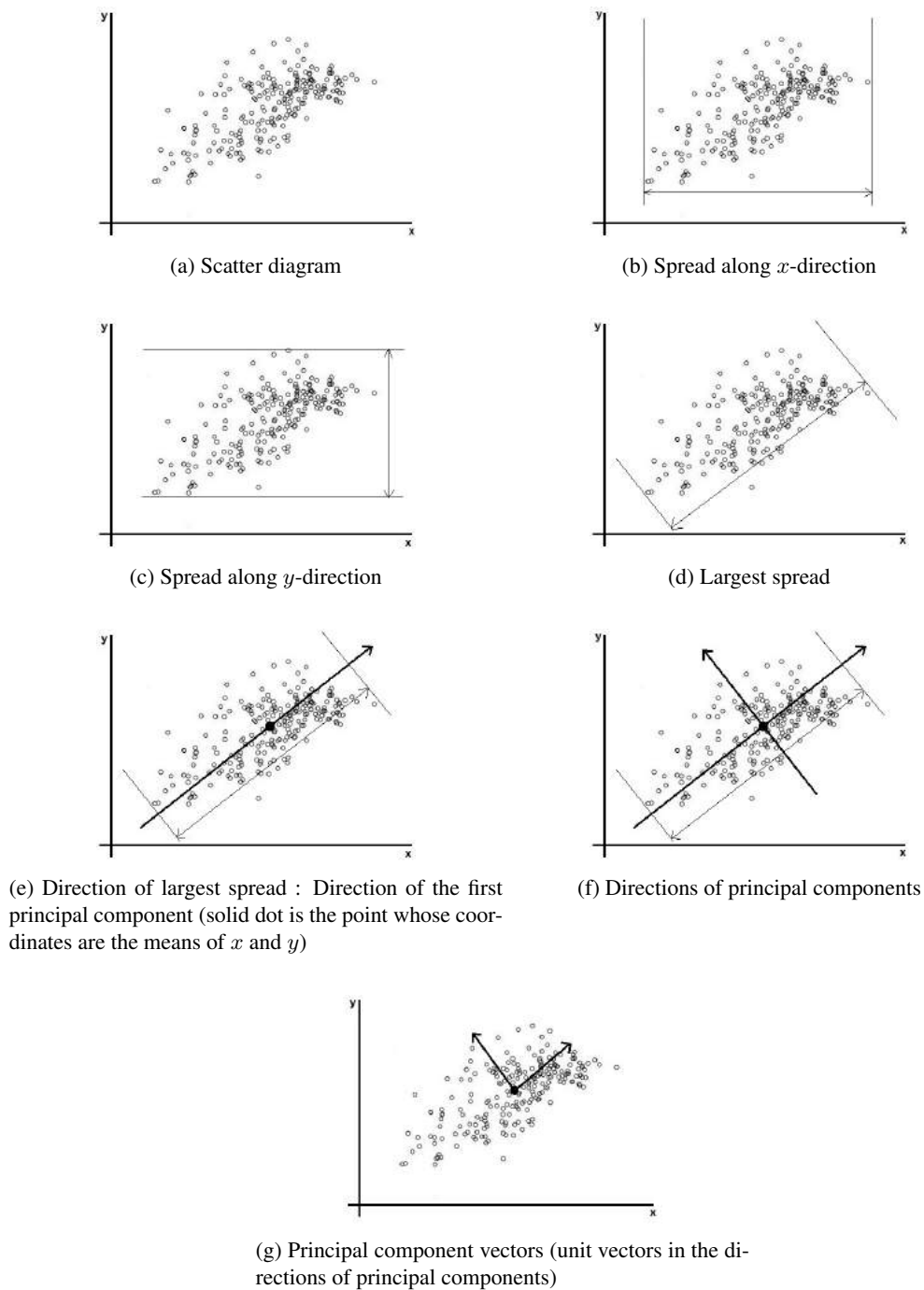


Figure 4.1: Principal components

A warning!

The graphical illustration of the idea of PCA as explained above is slightly misleading. For the sake of simplicity and easy geometrical representation, in the graphical illustration we have used *range* as the measure of spread. The direction of the first principal component was taken as the direction of maximum range. But, due to theoretical reasons, in the implementation of PCA in practice, it is the variance that is taken as the measure of spread. The first principal component is the direction in which the variance is maximum.

4.4.2 Computation of the principal component vectors (PCA algorithm)

The following is an outline of the procedure for performing a principal component analysis on a given data. The procedure is heavily dependent on mathematical concepts. A knowledge of these concepts is essential to carry out this procedure.

Step 1. Data

We consider a dataset having n features or variables denoted by $X_1, X_2, \dots, X_n$. Let there be N examples. Let the values of the i -th feature X_i be $X_{i1}, X_{i2}, \dots, X_{iN}$ (see Table 4.1).

Features	Example 1	Example 2	...	Example N
X_1	X_{11}	X_{12}	...	X_{1N}
X_2	X_{21}	X_{22}	...	X_{2N}
$\vdots$				
X_i	X_{i1}	X_{i2}	...	X_{iN}
$\vdots$				
X_n	X_{n1}	X_{n2}	...	X_{nN}

Table 4.1: Data for PCA algorithm

Step 2. Compute the means of the variables

We compute the mean $\bar{X}_i$ of the variable X_i :

$$\bar{X}_i = \frac{1}{N} (X_{i1} + X_{i2} + \dots + X_{iN}).$$

Step 3. Calculate the covariance matrix

Consider the variables X_i and X_j (i and j need not be different). The covariance of the ordered pair (X_i, X_j) is defined as¹

$$\text{Cov}(X_i, X_j) = \frac{1}{N-1} \sum_{k=1}^N (X_{ik} - \bar{X}_i)(X_{jk} - \bar{X}_j). \quad (4.1)$$

We calculate the following $n \times n$ matrix S called the covariance matrix of the data. The element in the i -th row j -th column is the covariance $\text{Cov}(X_i, X_j)$:

$$S = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) & \dots & \text{Cov}(X_1, X_n) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) & \dots & \text{Cov}(X_2, X_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_n, X_1) & \text{Cov}(X_n, X_2) & \dots & \text{Cov}(X_n, X_n) \end{bmatrix}$$

Step 4. Calculate the eigenvalues and eigenvectors of the covariance matrix

Let S be the covariance matrix and let I be the identity matrix having the same dimension as the dimension of S .

i) Set up the equation:

$$\det(S - \lambda I) = 0. \quad (4.2)$$

This is a polynomial equation of degree n in λ . It has n real roots (some of the roots may be repeated) and these roots are the eigenvalues of S . We find the n roots $\lambda_1, \lambda_2, \dots, \lambda_n$ of Eq. (4.2).

¹There is an alternative definition of covariance. In this definition, covariance is defined as in Eq. (4.1) with $N-1$ replaced by N . There are certain theoretical reasons for adopting the definition as given here.

- ii) If $\lambda = \lambda'$ is an eigenvalue, then the corresponding eigenvector is a vector

$$U = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}$$

such that

$$(S - \lambda'I)U = 0.$$

(This is a system of n homogeneous linear equations in $u_1, u_2, \dots, u_n$ and it always has a nontrivial solution.) We next find a set of n orthogonal eigenvectors $U_1, U_2, \dots, U_n$ such that U_i is an eigenvector corresponding to λ_i .²

- iii) We now normalise the eigenvectors. Given any vector X we normalise it by dividing X by its length. The length (or, the norm) of the vector

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

is defined as

$$\|X\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}.$$

Given any eigenvector U , the corresponding normalised eigenvector is computed as

$$\frac{1}{\|U\|}U.$$

We compute the n normalised eigenvectors $e_1, e_2, \dots, e_n$ by

$$e_i = \frac{1}{\|U_i\|}U_i, \quad i = 1, 2, \dots, n.$$

Step 5. Derive new data set

Order the eigenvalues from highest to lowest. The unit eigenvector corresponding to the largest eigenvalue is the first principal component. The unit eigenvector corresponding to the next highest eigenvalue is the second principal component, and so on.

- i) Let the eigenvalues in descending order be $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ and let the corresponding unit eigenvectors be $e_1, e_2, \dots, e_n$.
- ii) Choose a positive integer p such that $1 \leq p \leq n$.
- iii) Choose the eigenvectors corresponding to the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_p$ and form the following $p \times n$ matrix (we write the eigenvectors as row vectors):

$$F = \begin{bmatrix} e_1^T \\ e_2^T \\ \vdots \\ e_p^T \end{bmatrix},$$

where T in the superscript denotes the transpose.

²For $i \neq j$, the vectors U_i and U_j are orthogonal means $U_i^T U_j = 0$ where T denotes the transpose.

iv) We form the following $n \times N$ matrix:

$$X = \begin{bmatrix} X_{11} - \bar{X}_1 & X_{12} - \bar{X}_1 & \cdots & X_{1N} - \bar{X}_1 \\ X_{21} - \bar{X}_2 & X_{22} - \bar{X}_2 & \cdots & X_{2N} - \bar{X}_2 \\ \vdots & \vdots & \ddots & \vdots \\ X_{n1} - \bar{X}_n & X_{n2} - \bar{X}_n & \cdots & X_{nN} - \bar{X}_n \end{bmatrix}$$

v) Next compute the matrix:

$$X_{\text{new}} = FX.$$

Note that this is a $p \times N$ matrix. This gives us a dataset of N samples having p features.

Step 6. New dataset

The matrix X_{new} is the new dataset. Each row of this matrix represents the values of a feature. Since there are only p rows, the new dataset has only features.

Step 7. Conclusion

This is how the principal component analysis helps us in dimensional reduction of the dataset. Note that it is not possible to get back the original n -dimensional dataset from the new dataset.

4.4.3 Illustrative example

We illustrate the ideas of principal component analysis by considering a toy example. In the discussions below, all the details of the computations are given. This is to give the reader an idea of the complexity of computations and also to help the reader do a “worked example” by hand computations without recourse to software packages.

Problem

Given the data in Table 4.2, use PCA to reduce the dimension from 2 to 1.

Feature	Example 1	Example 2	Example 3	Example 4
X_1	4	8	13	7
X_2	11	4	5	14

Table 4.2: Data for illustrating PCA

Solution

1. Scatter plot of data

We have

$$\begin{aligned} \bar{X}_1 &= \frac{1}{4}(4 + 8 + 13 + 7) = 8, \\ \bar{X}_2 &= \frac{1}{4}(11 + 4 + 5 + 14) = 8.5. \end{aligned}$$

Figure 4.2 shows the scatter plot of the data together with the point $(\bar{X}_1, \bar{X}_2)$.

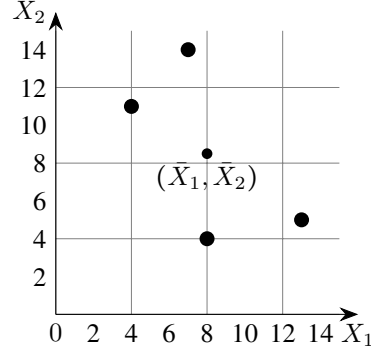


Figure 4.2: Scatter plot of data in Table 4.2

2. Calculation of the covariance matrix

The covariances are calculated as follows:

$$\begin{aligned}
 \text{Cov}(X_1, X_2) &= \frac{1}{N-1} \sum_{k=1}^N (X_{1k} - \bar{X}_1)^2 \\
 &= \frac{1}{3} ((4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2) \\
 &= 14 \\
 \text{Cov}(X_1, X_2) &= \frac{1}{N-1} \sum_{k=1}^N (X_{1k} - \bar{X}_1)(X_{2k} - \bar{X}_2) \\
 &= \frac{1}{3} ((4-8)(11-8.5) + (8-8)(4-8.5) \\
 &\quad + (13-8)(5-8.5) + (7-8)(14-8.5)) \\
 &= -11 \\
 \text{Cov}(X_2, X_1) &= \text{Cov}(X_1, X_2) \\
 &= -11 \\
 \text{Cov}(X_2, X_2) &= \frac{1}{N-1} \sum_{k=1}^N (X_{2k} - \bar{X}_2)^2 \\
 &= \frac{1}{3} ((11-8.5)^2 + (4-8.5)^2 + (5-8.5)^2 + (14-8.5)^2) \\
 &= 23
 \end{aligned}$$

The covariance matrix is

$$\begin{aligned}
 S &= \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) \end{bmatrix} \\
 &= \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}
 \end{aligned}$$

3. Eigenvalues of the covariance matrix

The characteristic equation of the covariance matrix is

$$\begin{aligned}
 0 &= \det(S - \lambda I) \\
 &= \begin{vmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{vmatrix} \\
 &= (14 - \lambda)(23 - \lambda) - (-11) \times (-11) \\
 &= \lambda^2 - 37\lambda + 201
 \end{aligned}$$

Solving the characteristic equation we get

$$\begin{aligned}\lambda &= \frac{1}{2}(37 \pm \sqrt{565}) \\ &= 30.3849, 6.6151 \\ &= \lambda_1, \lambda_2 \quad (\text{say})\end{aligned}$$

4. Computation of the eigenvectors

To find the first principal components, we need only compute the eigenvector corresponding to the largest eigenvalue. In the present example, the largest eigenvalue is λ_1 and so we compute the eigenvector corresponding to λ_1 .

The eigenvector corresponding to $\lambda = \lambda_1$ is a vector $U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$ satisfying the following equation:

$$\begin{aligned}\begin{bmatrix} 0 \\ 0 \end{bmatrix} &= (S - \lambda_1 I)X \\ &= \begin{bmatrix} 14 - \lambda_1 & -11 \\ -11 & 23 - \lambda_1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \\ &= \begin{bmatrix} (14 - \lambda_1)u_1 - 11u_2 \\ -11u_1 + (23 - \lambda_1)u_2 \end{bmatrix}\end{aligned}$$

This is equivalent to the following two equations:

$$\begin{aligned}(14 - \lambda_1)u_1 - 11u_2 &= 0 \\ -11u_1 + (23 - \lambda_1)u_2 &= 0\end{aligned}$$

Using the theory of systems of linear equations, we note that these equations are not independent and solutions are given by

$$\frac{u_1}{11} = \frac{u_2}{14 - \lambda_1} = t,$$

that is

$$u_1 = 11t, \quad u_2 = (14 - \lambda_1)t,$$

where t is any real number. Taking $t = 1$, we get an eigenvector corresponding to λ_1 as

$$U_1 = \begin{bmatrix} 11 \\ 14 - \lambda_1 \end{bmatrix}.$$

To find a unit eigenvector, we compute the length of U_1 which is given by

$$\begin{aligned}\|U_1\| &= \sqrt{11^2 + (14 - \lambda_1)^2} \\ &= \sqrt{11^2 + (14 - 30.3849)^2} \\ &= 19.7348\end{aligned}$$

Therefore, a unit eigenvector corresponding to λ_1 is

$$\begin{aligned}e_1 &= \begin{bmatrix} 11/\|U_1\| \\ (14 - \lambda_1)/\|U_1\| \end{bmatrix} \\ &= \begin{bmatrix} 11/19.7348 \\ (14 - 30.3849)/19.7348 \end{bmatrix} \\ &= \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}\end{aligned}$$

By carrying out similar computations, the unit eigenvector e_2 corresponding to the eigenvalue $\lambda = \lambda_2$ can be shown to be

$$e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix}.$$

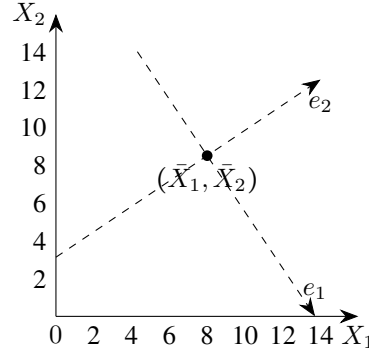


Figure 4.3: Coordinate system for principal components

5. Computation of first principal components

Let $\begin{bmatrix} X_{1k} \\ X_{2k} \end{bmatrix}$ be the k -th sample in Table 4.2. The first principal component of this example is given by (here “ T ” denotes the transpose of the matrix)

$$\begin{aligned} e_1^T \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix} &= \begin{bmatrix} 0.5574 & -0.8303 \end{bmatrix} \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix} \\ &= 0.5574(X_{1k} - \bar{X}_1) - 0.8303(X_{2k} - \bar{X}_2). \end{aligned}$$

For example, the first principal component corresponding to the first example $\begin{bmatrix} X_{11} \\ X_{21} \end{bmatrix} = \begin{bmatrix} 4 \\ 11 \end{bmatrix}$ is calculated as follows:

$$\begin{aligned} \begin{bmatrix} 0.5574 & -0.8303 \end{bmatrix} \begin{bmatrix} X_{11} - \bar{X}_1 \\ X_{21} - \bar{X}_2 \end{bmatrix} &= 0.5574(X_{11} - \bar{X}_1) - 0.8303(X_{21} - \bar{X}_2) \\ &= 0.5574(4 - 8) - 0.8303(11 - 8, 5) \\ &= -4.30535 \end{aligned}$$

The results of calculations are summarised in Table 4.3.

X_1	4	8	13	7
X_2	11	4	5	14
First principal components	-4.3052	3.7361	5.6928	-5.1238

Table 4.3: First principal components for data in Table 4.2

6. Geometrical meaning of first principal components

As we have seen in Figure 4.1, we introduce new coordinate axes. First we shift the origin to the “center” $(\bar{X}_1, \bar{X}_2)$ and then change the directions of coordinate axes to the directions of the eigenvectors e_1 and e_2 (see Figure 4.3).

Next, we drop perpendiculars from the given data points to the e_1 -axis (see Figure 4.4). The first principal components are the e_1 -coordinates of the feet of perpendiculars, that is, the projections on the e_1 -axis. The projections of the data points on e_1 -axis may be taken as approximations of the given data points hence we may replace the given data set with these points. Now, each of these

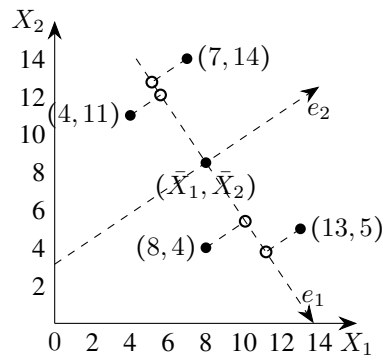


Figure 4.4: Projections of data points on the axis of the first principal component

PC1 components	-4.305187	3.736129	5.692828	-5.123769
----------------	-----------	----------	----------	-----------

Table 4.4: One-dimensional approximation to the data in Table 4.2

approximations can be unambiguously specified by a single number, namely, the e_1 -coordinate of approximation. Thus the two-dimensional data set given in Table 4.2 can be represented approximately by the following one-dimensional data set (see Figure 4.5):

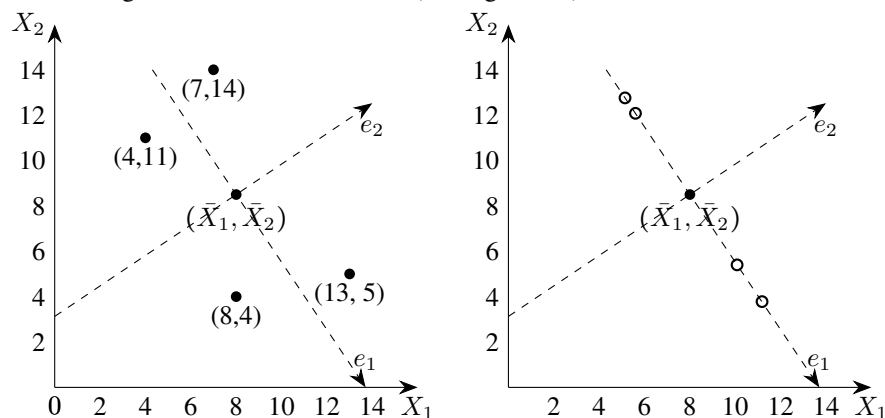


Figure 4.5: Geometrical representation of one-dimensional approximation to the data in Table 4.2

4.5 Sample questions

(a) Short answer questions

1. What is dimensionality reduction? How is it implemented?
2. Explain why dimensionality reduction is useful in machine learning.
3. What are the commonly used dimensionality reduction techniques in machine learning?
4. How is the subset selection method used for dimensionality reduction?
5. Explain the method of principal component analysis in machine learning.
6. What are the first principal components of a data?
7. Is subset selection problem an unsupervised learning problem? Why?

8. Is principal component analysis a supervised learning problem? Why?

(b) Long answer questions

1. Describe the forward selection algorithm for implementing the subset selection procedure for dimensionality reduction.
2. Describe the backward selection algorithm for implementing the subset selection procedure for dimensionality reduction.
3. What is the first principal component of a data? How one can compute it?
4. Describe with the use of diagrams the basic principle of PCA.
5. Explain the procedure for the computation of the principal components of a given data.
6. Describe how principal component analysis is carried out to reduce dimensionality of data sets.
7. Given the following data, compute the principal component vectors and the first principal components:

x	2	3	7
y	11	14	26

8. Given the following data, compute the principal component vectors and the first principal components:

x	-3	1	-2
y	2	-1	3

Chapter 5

Evaluation of classifiers

In machine learning, there are several classification algorithms and, given a certain problem, more than one may be applicable. So there is a need to examine how we can assess how good a selected algorithm is. Also, we need a method to compare the performance of two or more different classification algorithms. These methods help us choose the right algorithm in a practical situation.

5.1 Methods of evaluation

5.1.1 Need for multiple validation sets

When we apply a classification algorithm in a practical situation, we always do a validation test. We keep a small sample of examples as validation set and the remaining set as the training set. The classifier developed using the training set is applied to the examples in the validation set. Based on the performance on the validation set, the accuracy of the classifier is assessed. But, the performance measure obtained by a single validation set alone does not give a true picture of the performance of a classifier. Also these measures alone cannot be meaningfully used to compare two algorithms. This requires us to have different validation sets.

Cross-validation in general, and k -fold cross-validation in particular, are two common method for generating multiple training-validation sets from a given dataset.

5.1.2 Statistical distribution of errors

We use a classification algorithm on a dataset and generate a classifier. If we do the training once, we have one classifier and one validation error. To average over randomness (in training data, initial weights, etc.), we use the same algorithm and generate multiple classifiers. We test these classifiers on multiple validation sets and record a sample of validation errors. We base our evaluation of the classification algorithm on the *statistical distribution of these validation errors*. We can use this distribution for assessing the *expected error* rate of the classification algorithm for that problem, or compare it with the error rate distribution of some other classification algorithm.

A detailed discussion of these ideas is beyond the scope of these notes.

5.1.3 No-free lunch theorem

Whatever conclusion we draw from our analysis is conditioned on the dataset we are given. We are not comparing classification algorithms in a domain-independent way but on some particular application. We are not saying anything about the expected error-rate of a learning algorithm, or comparing one learning algorithm with another algorithm, in general. Any result we have is only true for the particular application. There is no such thing as the “best” learning algorithm. For any

learning algorithm, there is a dataset where it is very accurate and another dataset where it is very poor. This is called the *No Free Lunch Theorem*.¹

5.1.4 Other factors

Classification algorithms can be compared based not only on error rates but also on several other criteria like the following:

- risks when errors are generalized using loss functions
- training time and space complexity,
- testing time and space complexity,
- interpretability, namely, whether the method allows knowledge extraction which can be checked and validated by experts, and
- easy programmability.

5.2 Cross-validation

To test the performance of a classifier, we need to have a number of training/validation set pairs from a dataset X . To get them, if the sample X is large enough, we can randomly divide it then divide each part randomly into two and use one half for training and the other half for validation. Unfortunately, datasets are never large enough to do this. So, we use the same data split differently; this is called *cross-validation*.

Cross-validation is a technique to evaluate predictive models by partitioning the original sample into a training set to train the model, and a test set to evaluate it.

The *holdout method* is the simplest kind of cross validation. The data set is separated into two sets, called the training set and the testing set. The algorithm fits a function using the training set only. Then the function is used to predict the output values for the data in the testing set (it has never seen these output values before). The errors it makes are used to evaluate the model.

5.3 K -fold cross-validation

In K -fold cross-validation, the dataset X is divided randomly into K equal-sized parts, X_i , $i = 1, \dots, K$. To generate each pair, we keep one of the K parts out as the validation set V_i , and combine the remaining $K - 1$ parts to form the training set T_i . Doing this K times, each time leaving out another one of the K parts out, we get K pairs (V_i, T_i) :

$$\begin{aligned} V_1 &= X_1, & T_1 &= X_2 \cup X_3 \cup \dots \cup X_K \\ V_2 &= X_2, & T_2 &= X_1 \cup X_3 \cup \dots \cup X_K \\ &\dots & & \\ V_K &= X_K, & T_K &= X_1 \cup X_2 \cup \dots \cup X_{K-1} \end{aligned}$$

Remarks

1. There are two problems with this: First, to keep the training set large, we allow validation sets that are small. Second, the training sets overlap considerably, namely, any two training sets share $K - 2$ parts.

¹“We have dubbed the associated results NFL theorems because they demonstrate that if an algorithm performs well on a certain class of problems then it necessarily pays for that with degraded performance on the set of all remaining problems.”(David Wolpert and William Macready in [7])

2. K is typically 10 or 30. As K increases, the percentage of training instances increases and we get more robust estimators, but the validation set becomes smaller. Furthermore, there is the cost of training the classifier K times, which increases as K is increased.

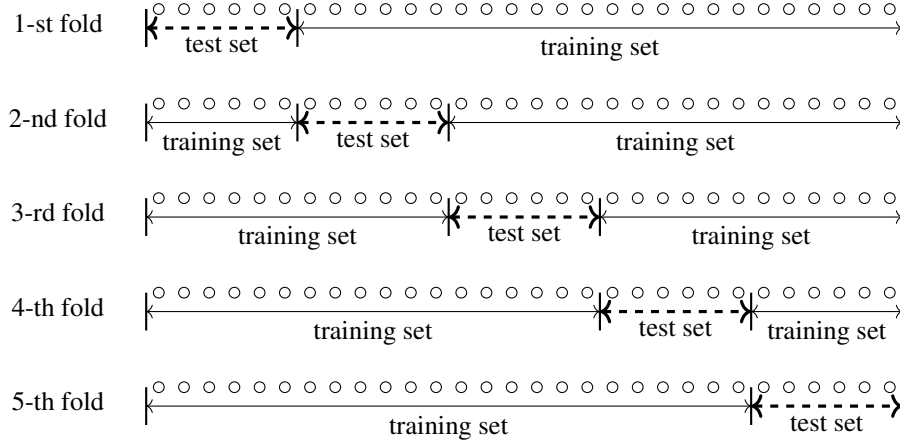


Figure 5.1: One iteration in a 5-fold cross-validation

Leave-one-out cross-validation

An extreme case of K -fold cross-validation is *leave-one-out* where given a dataset of N instances, only one instance is left out as the validation set and training uses the remaining $N - 1$ instances. We then get N separate pairs by leaving out a different instance at each iteration. This is typically used in applications such as medical diagnosis, where labeled data is hard to find.

5.3.1 5×2 cross-validation

In this method, the dataset X is divided into two equal parts $X_1^{(1)}$ and $X_1^{(2)}$. We take as the training set and $X_1^{(2)}$ as the validation set. We then swap the two sets and take $X_1^{(2)}$ as the training set and $X_1^{(1)}$ as the validation set. This is the first fold. the process is repeated four more times to get ten pairs of training sets and validation sets.

$$\begin{aligned}
 T_1 &= X_1^{(1)}, & V_1 &= X_1^{(2)} \\
 T_2 &= X_1^{(2)}, & V_2 &= X_1^{(1)} \\
 T_3 &= X_2^{(1)}, & V_3 &= X_2^{(2)} \\
 T_4 &= X_2^{(2)}, & V_4 &= X_2^{(1)} \\
 &\vdots & & \\
 T_9 &= X_5^{(1)}, & V_9 &= X_5^{(2)} \\
 T_{10} &= X_5^{(2)}, & V_{10} &= X_5^{(1)}
 \end{aligned}$$

It has been shown that after five folds, the validation error rates become too dependent and do not add new information. On the other hand, if there are fewer than five folds, we get fewer data (fewer than ten) and will not have a large enough sample to fit a distribution and test our hypothesis.

5.3.2 Bootstrapping

Bootstrapping in statistics

In statistics, the term “bootstrap sampling”, the “bootstrap” or “bootstrapping” for short, refers to process of “random sampling with replacement”.

Example

For example, let there be five balls labeled A, B, C, D, E in an urn. We wish to select different samples of balls from the urn each sample containing two balls. The following procedure may be used to select the samples. This is an example for bootstrap sampling.

1. We select two balls from the basket. Let them be A and E. Record the labels.
2. Put the two balls back in the basket.
3. We select two balls from the basket. Let them be C and E. Record the labels.
4. Put the two balls back into the basket.

This is repeated as often as required. So we get different samples of size 2, say, A, E; B, E; etc. These samples are obtained by sampling with replacement, that is, by bootstrapping.

Bootstrapping in machine learning

In machine learning, bootstrapping is the process of computing performance measures using several randomly selected training and test datasets which are selected through a process of sampling with replacement, that is, through bootstrapping. Sample datasets are selected multiple times.

The bootstrap procedure will create one or more new training datasets some of which are repeated. The corresponding test datasets are then constructed from the set of examples that were not selected for the respective training datasets.

5.4 Measuring error

5.4.1 True positive, false positive, etc.

Definitions

Consider a binary classification model derived from a two-class dataset. Let the class labels be c and $\neg c$. Let x be a test instance.

1. True positive

Let the true class label of x be c . If the model predicts the class label of x as c , then we say that the classification of x is *true positive*.

2. False negative

Let the true class label of x be c . If the model predicts the class label of x as $\neg c$, then we say that the classification of x is *false negative*.

3. True negative

Let the true class label of x be $\neg c$. If the model predicts the class label of x as $\neg c$, then we say that the classification of x is *true negative*.

4. False positive

Let the true class label of x be $\neg c$. If the model predicts the class label of x as c , then we say that the classification of x is *false positive*.

	Actual label of x is c	Actual label of x is $\neg c$
Predicted label of x is c	True positive	False positive
Predicted label of x is $\neg c$	False negative	True negative

5.4.2 Confusion matrix

A confusion matrix is used to describe the performance of a classification model (or “classifier”) on a set of test data for which the true values are known. A *confusion matrix* is a table that categorizes predictions according to whether they match the actual value.

Two-class datasets

For a two-class dataset, a confusion matrix is a table with two rows and two columns that reports the number of false positives, false negatives, true positives, and true negatives.

Assume that a classifier is applied to a two-class test dataset for which the true values are known. Let TP denote the number of true positives in the predicted values, TN the number of true negatives, etc. Then the confusion matrix of the predicted values can be represented as follows:

	Actual condition is true	Actual condition is false
Predicted condition is true	TP	FP
Predicted condition is false	FN	TN

Table 5.1: Confusion matrix for two-class dataset

Multiclass datasets

Confusion matrices can be constructed for multiclass datasets also.

Example

If a classification system has been trained to distinguish between cats, dogs and rabbits, a confusion matrix will summarize the results of testing the algorithm for further inspection. Assuming a sample of 27 animals - 8 cats, 6 dogs, and 13 rabbits, the resulting confusion matrix could look like the table below: This confusion matrix shows that, for example, of the 8 actual cats, the system predicted that

	Actual “cat”	Actual “dog”	Actual “rabbit”
Predicted “cat”	5	2	0
Predicted “dog”	3	3	2
Predicted “rabbit”	0	1	11

three were dogs, and of the six dogs, it predicted that one was a rabbit and two were cats.

5.4.3 Precision and recall

In machine learning, precision and recall are two measures used to assess the quality of results produced by a binary classifier. They are formally defined as follows.

Definitions

Let a binary classifier classify a collection of test data. Let

TP = Number of true positives

TN = Number of true negatives

FP = Number of false positives

FN = Number of false negatives

The *precision* P is defined as

$$P = \frac{TP}{TP + FP}$$

The *recall* R is defined as

$$R = \frac{TP}{TP + FN}$$

Problem 1

Suppose a computer program for recognizing dogs in photographs identifies eight dogs in a picture containing 12 dogs and some cats. Of the eight dogs identified, five actually are dogs while the rest are cats. Compute the precision and recall of the computer program.

Solution

We have:

$$TP = 5$$

$$FP = 3$$

$$FN = 7$$

The *precision* P is

$$P = \frac{TP}{TP + FP} = \frac{5}{5 + 3} = \frac{5}{8}$$

The *recall* R is

$$R = \frac{TP}{TP + FN} = \frac{5}{5 + 7} = \frac{5}{12}$$

Problem 2

Let there be 10 balls (6 white and 4 red balls) in a box and let it be required to pick up the red balls from them. Suppose we pick up 7 balls as the red balls of which only 2 are actually red balls. What are the values of precision and recall in picking red ball?

Solution

Obviously we have:

$$TP = 2$$

$$FP = 7 - 2 = 5$$

$$FN = 4 - 2 = 2$$

The *precision* P is

$$P = \frac{TP}{TP + FP} = \frac{2}{2 + 5} = \frac{2}{7}$$

The *recall* R is

$$R = \frac{TP}{TP + FN} = \frac{2}{2 + 2} = \frac{1}{2}$$

Problem 3

Assume the following: A database contains 80 records on a particular topic of which 55 are relevant to a certain investigation. A search was conducted on that topic and 50 records were retrieved. Of the 50 records retrieved, 40 were relevant. Construct the confusion matrix for the search and calculate the precision and recall scores for the search.

Solution

Each record may be assigned a class label “relevant” or “not relevant”. All the 80 records were tested for relevance. The test classified 50 records as “relevant”. But only 40 of them were actually relevant. Hence we have the following confusion matrix for the search:

	Actual “relevant”	Actual “not relevant”
Predicted “relevant”	40	10
Predicted “not relevant”	15	25

Table 5.2: Example for confusion matrix

$$TP = 40$$

$$FP = 10$$

$$FN = 15$$

The *precision* P is

$$P = \frac{TP}{TP + FP} = \frac{40}{40 + 10} = \frac{4}{5}$$

The *recall* R is

$$R = \frac{TP}{TP + FN} = \frac{40}{40 + 15} = \frac{40}{55}$$

5.4.4 Other measures of performance

Using the data in the confusion matrix of a classifier of two-class dataset, several measures of performance have been defined. A few of them are listed below.

$$1. \text{ Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$2. \text{ Error rate} = 1 - \text{Accuracy}$$

$$3. \text{ Sensitivity} = \frac{TP}{TP + FN}$$

$$4. \text{ Specificity} = \frac{TN}{TN + FP}$$

$$5. \text{ F-measure} = \frac{2 \times TP}{2 \times TP + FP + FN}$$

5.5 Receiver Operating Characteristic (ROC)

The acronym ROC stands for Receiver Operating Characteristic, a terminology coming from signal detection theory. The ROC curve was first developed by electrical engineers and radar engineers during World War II for detecting enemy objects in battlefields. They are now increasingly used in machine learning and data mining research.

TPR and FPR

Let a binary classifier classify a collection of test data. Let, as before,

TP = Number of true positives

TN = Number of true negatives

FP = Number of false positives

FN = Number of false negatives

Now we introduce the following terminology:

TPR = True Positive Rate

$$\begin{aligned} &= \frac{TP}{TP + FN} \\ &= \text{Fraction of positive examples correctly classified} \\ &= \text{Sensitivity} \end{aligned}$$

FPR = False Positive Rate

$$\begin{aligned} &= \frac{FP}{FP + TN} \\ &= \text{Fraction of negative examples incorrectly classified} \\ &= 1 - \text{Specificity} \end{aligned}$$

ROC space

We plot the values of FPR along the horizontal axis (that is, x -axis) and the values of TPR along the vertical axis (that is, y -axis) in a plane. For each classifier, there is a unique point in this plane with coordinates (FPR, TPR). The ROC space is the part of the plane whose points correspond to (FPR, TPR). Each prediction result or instance of a confusion matrix represents one point in the ROC space.

The position of the point (FPR, TPR) in the ROC space gives an indication of the performance of the classifier. For example, let us consider some special points in the space.

Special points in ROC space**1. The left bottom corner point (0, 0): Always negative prediction**

A classifier which produces this point in the ROC space never classifies an example as positive, neither rightly nor wrongly, because for this point $TP = 0$ and $FP = 0$. It always makes negative predictions. All positive instances are wrongly predicted and all negative instances are correctly predicted. It commits no false positive errors.

2. The right top corner point (1, 1): Always positive prediction

A classifier which produces this point in the ROC space always classifies an example as positive because for this point $FN = 0$ and $TN = 0$. All positive instances are correctly predicted and all negative instances are wrongly predicted. It commits no false negative errors.

3. The left top corner point (0, 1): Perfect prediction

A classifier which produces this point in the ROC space may be thought as a perfect classifier. It produces no false positives and no false negatives.

4. Points along the diagonal: Random performance

Consider a classifier where the class labels are randomly guessed, say by flipping a coin. Then, the corresponding points in the ROC space will be lying very near the diagonal line joining the points (0, 0) and (1, 1).

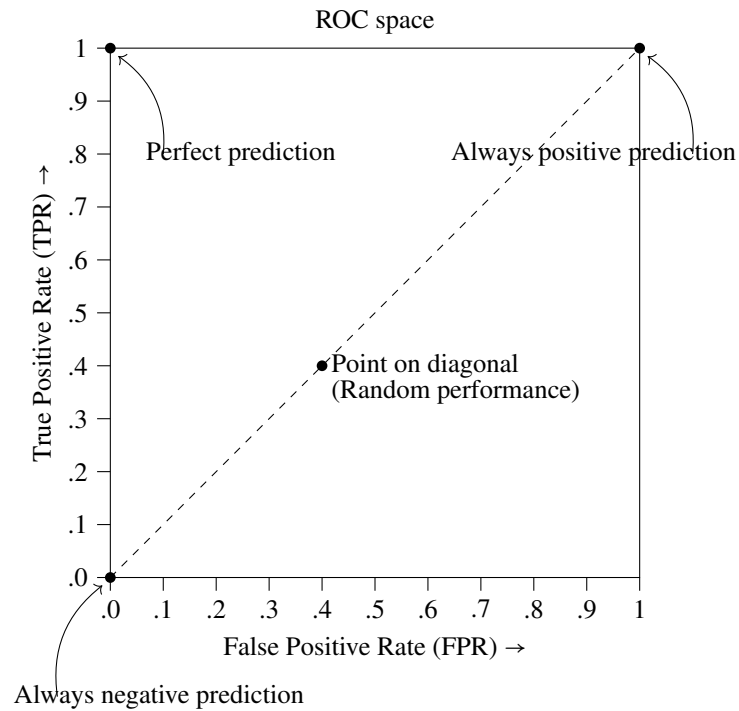


Figure 5.2: The ROC space and some special points in the space

ROC curve

In the case of certain classification algorithms, the classifier may depend on a parameter. Different values of the parameter will give different classifiers and these in turn give different values to TPR and FPR. The ROC curve is the curve obtained by plotting in the ROC space the points (TPR, FPR) obtained by assigning all possible values to the parameter in the classifier.

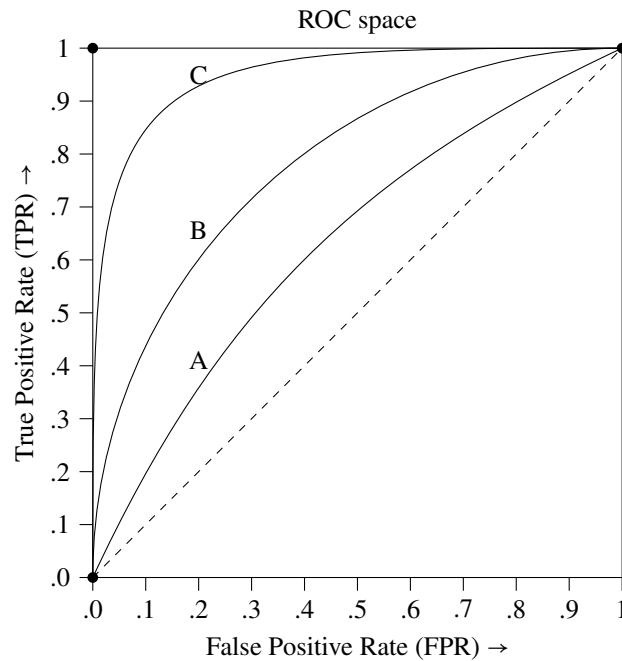


Figure 5.3: ROC curves of three different classifiers A, B, C

The closer the ROC curve is to the top left corner $(0, 1)$ of the ROC space, the better the accuracy of the classifier. Among the three classifiers A, B, C with ROC curves as shown in Figure 5.3, the classifier C is closest to the top left corner of the ROC space. Hence, among the three, it gives the best accuracy in predictions.

Example

Cut-off value of BMI	Breast cancer		Normal persons		TPR	FPR
	TP	FN	FP	TN		
18	100	0	200	0	1.00	1.000
20	100	0	198	2	1.00	0.990
22	99	1	177	23	0.99	0.885
24	95	5	117	83	0.95	0.585
26	85	15	80	120	0.85	0.400
28	66	34	53	147	0.66	0.265
30	47	53	27	173	0.47	0.135
32	34	66	17	183	0.34	0.085
34	21	79	14	186	0.21	0.070
36	17	83	6	194	0.17	0.030
38	7	93	4	196	0.07	0.020
40	1	99	1	199	0.01	0.005

Table 5.3: Data on breast cancer for various values of BMI

The body mass index (BMI) of a person is defined as $(\text{weight}(\text{kg})/\text{height}(\text{m})^2)$. Researchers have established a link between BMI and the risk of breast cancer among women. The higher the BMI the higher the risk of developing breast cancer. The critical threshold value of BMI may depend on several parameters like food habits, socio-cultural-economic background, life-style, etc. Table 5.3

gives real data of a breast cancer study with a sample having 100 patients and 200 normal persons.² The table also shows the values of TPR and FPR for various cut-off values of BMI. The ROC curve of the data in Table 5.3 is shown in Figure 5.4.

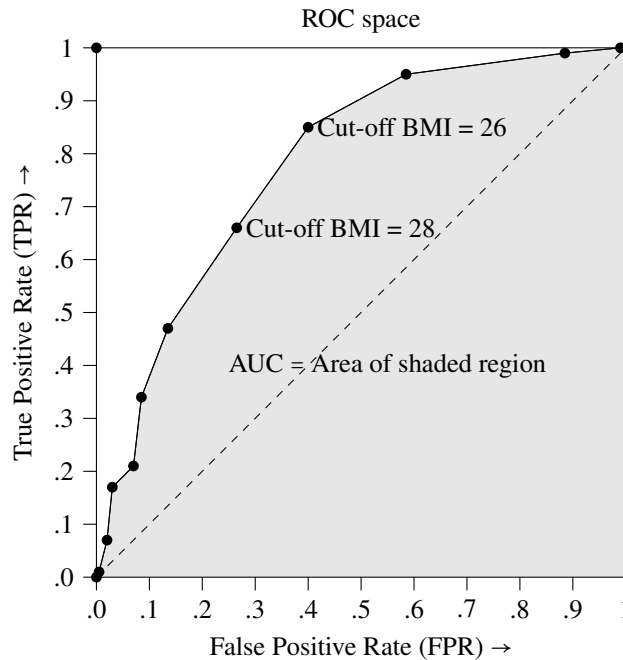


Figure 5.4: ROC curve of data in Table 5.3 showing the points closest to the perfect prediction point (0, 1)

Area under the ROC curve

The measure of the area under the ROC curve is denoted by the acronym AUC (see Figure 5.4). The value of AUC is a measure of the performance of a classifier. For the perfect classifier, $AUC = 1.0$.

5.6 Sample questions

(a) Short answer questions

1. What is cross-validation in machine learning?
2. What is meant by 5×2 cross-validation?
3. What is meant by leave-one-out cross validation?
4. What is meant by the confusion matrix of a binary classification problem.
5. Define the following terms: precision, recall, sensitivity, specificity.
6. What is ROC curve in machine learning?
7. What are true positive rates and false positive rates in machine learning?
8. What is AUC in relation to ROC curves?

²<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3755824/>

(b) Long answer questions

1. Explain cross-validation in machine learning. Explain the different types of cross-validations.
2. What is meant by true positives etc.? What is meant by confusion matrix of a binary classification problem? Explain how this can be extended to multi-class problems.
3. What are ROC space and ROC curve in machine learning? In ROC space, which points correspond to perfect prediction, always positive prediction and always negative prediction? Why?
4. Consider a two-class classification problem of predicting whether a photograph contains a man or a woman. Suppose we have a test dataset of 10 records with expected outcomes and a set of predictions from our classification algorithm.

	Expected	Predicted
1	man	woman
2	man	man
3	woman	woman
4	man	man
5	woman	man
6	woman	woman
7	woman	woman
8	man	man
9	man	woman
10	woman	woman

- (a) Compute the confusion matrix for the data.
 - (b) Compute the accuracy, precision, recall, sensitivity and specificity of the data.
5. Suppose 10000 patients get tested for flu; out of them, 9000 are actually healthy and 1000 are actually sick. For the sick people, a test was positive for 620 and negative for 380. For the healthy people, the same test was positive for 180 and negative for 8820. Construct a confusion matrix for the data and compute the accuracy, precision and recall for the data.
 6. Given the following data, construct the ROC curve of the data. Compute the AUC.

Threshold	TP	TN	FP	FN
1	0	25	0	29
2	7	25	0	22
3	18	24	1	11
4	26	20	5	3
5	29	11	14	0
6	29	0	25	0
7	29	0	25	0

7. Given the following hypothetical data at various cut-off points of mid-arm circumference of mid-arm circumference to detect low birth-weight construct the ROC curve for the data.

Mid-arm circumference (cm)	Normal birth-weight TP	Low birth-weight TN
≤ 8.3	13	867
≤ 8.4	24	844
≤ 8.5	73	826
≤ 8.6	90	800
≤ 8.7	113	783
≤ 8.8	119	735
≤ 8.9	121	626
≤ 9.0	125	505
≤ 9.1	127	435
≤ 9.2 and above	130	0

Chapter 6

Bayesian classifier and ML estimation

The Bayesian classifier is an algorithm for classifying multiclass datasets. This is based on the Bayes' theorem in probability theory. Bayes in whose name the theorem is known was an English statistician who was known for having formulated a specific case of a theorem that bears his name. The classifier is also known as “naive Bayes Algorithm” where the word “naive” is an English word with the following meanings: simple, unsophisticated, or primitive. We first explain Bayes' theorem and then describe the algorithm. Of course, we require the notion of conditional probability.

6.1 Conditional probability

The probability of the occurrence of an event A given that an event B has already occurred is called the *conditional probability of A given B* and is denoted by $P(A|B)$. We have

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad \text{if } P(B) \neq 0.$$

6.1.1 Independent events

1. Two events A and B are said to be *independent* if

$$P(A \cap B) = P(A)P(B).$$

2. Three events A, B, C are said to be *pairwise independent* if

$$P(B \cap C) = P(B)P(C)$$

$$P(C \cap A) = P(C)P(A)$$

$$P(A \cap B) = P(A)P(B)$$

3. Three events A, B, C are said to be *mutually independent* if

$$P(B \cap C) = P(B)P(C) \tag{6.1}$$

$$P(C \cap A) = P(C)P(A) \tag{6.2}$$

$$P(A \cap B) = P(A)P(B) \tag{6.3}$$

$$P(A \cap B \cap C) = P(A)P(B)P(C) \tag{6.4}$$

4. In general, a family of k events $A_1, A_2, \dots, A_k$ is said to be mutually independent if for any subfamily consisting of $A_{i_1}, \dots, A_{i_m}$ we have

$$P(A_{i_1} \cap \dots \cap A_{i_m}) = P(A_{i_1}) \dots P(A_{i_m}).$$

Remarks

Consider events and respective probabilities as shown in Figure 6.1. It can be seen that, in this case, the conditions Eqs.(6.1)–(6.3) are satisfied, but Eq.(6.4) is not satisfied. But if the probabilities are as in Figure 6.2, then Eq.(6.4) is satisfied but all the conditions in Eqs.(6.1)–(6.2) are not satisfied.

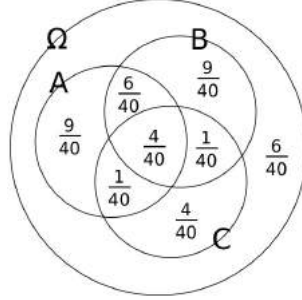


Figure 6.1: Events A, B, C which are not mutually independent: Eqs.(6.1)–(6.3) are satisfied, but Eq.(6.4) is not satisfied.

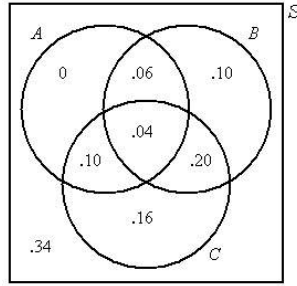


Figure 6.2: Events A, B, C which are not mutually independent: Eq.(6.4) is satisfied but Eqs.(6.1)–(6.2) are not satisfied.

6.2 Bayes' theorem

6.2.1 Theorem

Let A and B any two events in a random experiment. If $P(A) \neq 0$, then

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}.$$

6.2.2 Remarks

1. The importance of the result is that it helps us to “invert” conditional probabilities, that is, to express the conditional probability $P(A|B)$ in terms of the conditional probability $P(B|A)$.
2. The following terminology is used in this context:
 - A is called the *proposition* and B is called the *evidence*.
 - $P(A)$ is called the *prior probability* of proposition and $P(B)$ is called the prior probability of evidence.

- $P(A|B)$ is called the *posterior probability* of A given B .
- $P(B|A)$ is called the *likelihood* of B given A .

6.2.3 Generalisation

Let the sample space be divided into disjoint events $B_1, B_2, \dots, B_n$ and A be any event. Then we have

$$P(B_k|A) = \frac{P(A|B_k)P(B_k)}{\sum_{i=1}^n P(A|B_i)P(B_i)}$$

6.2.4 Examples

Problem 1

Consider a set of patients coming for treatment in a certain clinic. Let A denote the event that a “Patient has liver disease” and B the event that a “Patient is an alcoholic.” It is known from experience that 10% of the patients entering the clinic have liver disease and 5% of the patients are alcoholics. Also, among those patients diagnosed with liver disease, 7% are alcoholics. Given that a patient is alcoholic, what is the probability that he will have liver disease?

Solution

Using the notations of probability, we have

$$\begin{aligned} P(A) &= 10\% = 0.10 \\ P(B) &= 5\% = 0.05 \\ P(B|A) &= 7\% = 0.07 \\ P(A|B) &= \frac{P(B|A)P(A)}{P(B)} \\ &= \frac{0.07 \times 0.10}{0.05} \\ &= 0.14 \end{aligned}$$

Problem 2

Three factories A, B, C of an electric bulb manufacturing company produce respectively 35%, 35% and 30% of the total output. Approximately 1.5%, 1% and 2% of the bulbs produced by these factories are known to be defective. If a randomly selected bulb manufactured by the company was found to be defective, what is the probability that the bulb was manufactured in factory A?

Solution

Let A, B, C denote the events that a randomly selected bulb was manufactured in factory A, B, C respectively. Let D denote the event that a bulb is defective. We have the following data:

$$\begin{aligned} P(A) &= 0.35, & P(B) &= 0.35, & P(C) &= 0.30 \\ P(D|A) &= 0.015, & P(D|B) &= 0.010, & P(D|C) &= 0.020 \end{aligned}$$

We are required to find $P(A|D)$. By the generalisation of the Bayes’ theorem we have:

$$\begin{aligned} P(A|D) &= \frac{P(D|A)P(A)}{P(D|A)P(A) + P(D|B)P(B) + P(D|C)P(C)} \\ &= \frac{0.015 \times 0.35}{0.015 \times 0.35 + 0.010 \times 0.35 + 0.020 \times 0.30} \\ &= 0.356. \end{aligned}$$

6.3 Naive Bayes algorithm

6.3.1 Assumption

The naive Bayes algorithm is based on the following assumptions:

- All the *features are independent* and are unrelated to each other. Presence or absence of a feature does not influence the presence or absence of any other feature.
- The data has *class-conditional independence*, which means that events are independent so long as they are conditioned on the same class value.

These assumptions are, in general, true in many real world problems. It is because of these assumptions, the algorithm is called a *naive* algorithm.

6.3.2 Basic idea

Suppose we have a training data set consisting of N examples having n features. Let the features be named as $(F_1, \dots, F_n)$. A feature vector is of the form $(f_1, f_2, \dots, f_n)$. Associated with each example, there is a certain class label. Let the set of class labels be $\{c_1, c_2, \dots, c_p\}$.

Suppose we are given a test instance having the feature vector

$$X = (x_1, x_2, \dots, x_n).$$

We are required to determine the most appropriate class label that should be assigned to the test instance. For this purpose we compute the following conditional probabilities

$$P(c_1|X), P(c_2|X), \dots, P(c_p|X). \quad (6.5)$$

and choose the maximum among them. Let the maximum probability be $P(c_i|X)$. Then, we choose c_i as the most appropriate class label for the training instance having X as the feature vector.

The direct computation of the probabilities given in Eq.(6.5) are difficult for a number of reasons. The Bayes' theorem can be applied to obtain a simpler method. This is explained below.

6.3.3 Computation of probabilities

Using Bayes' theorem, we have:

$$P(c_k|X) = \frac{P(X|c_k)P(c_k)}{P(X)} \quad (6.6)$$

Since, by assumption, the data has class-conditional independence, we note that the events " $x_1|c_k$ ", " $x_2|c_k$ ", ..., " $x_n|c_k$ " are independent (because they are all conditioned on the same class label c_k). Hence we have

$$\begin{aligned} P(X|c_k) &= P((x_1, x_2, \dots, x_n)|c_k) \\ &= P(x_1|c_k)P(x_2|c_k) \cdots P(x_n|c_k) \end{aligned}$$

Using this in Eq.(6.6) we get

$$P(c_k|X) = \frac{P(x_1|c_k)P(x_2|c_k) \cdots P(x_n|c_k)P(c_k)}{P(X)}.$$

Since the denominator $P(X)$ is independent of the class labels, we have

$$P(c_k|X) \propto P(x_1|c_k)P(x_2|c_k) \cdots P(x_n|c_k)P(c_k).$$

So it is enough to find the maximum among the following values:

$$P(x_1|c_k)P(x_2|c_k) \cdots P(x_n|c_k)P(c_k), \quad k = 1, \dots, p.$$

Remarks

The various probabilities in the above expression are computed as follows:

$$P(c_k) = \frac{\text{No. of examples with class label } c_k}{\text{Total number of examples}}$$

$$P(x_j|c_k) = \frac{\text{No. of examples with } j\text{th feature equal to } x_j \text{ and class label } c_k}{\text{No. of examples with class label } c_k}$$

6.3.4 The algorithm**Algorithm: Naive Bayes**

Let there be a training data set having n features $F_1, \dots, F_n$. Let f_1 denote an arbitrary value of F_1 , f_2 of F_2 , and so on. Let the set of class labels be $\{c_1, c_2, \dots, c_p\}$. Let there be given a test instance having the feature vector

$$X = (x_1, x_2, \dots, x_n).$$

We are required to determine the most appropriate class label that should be assigned to the test instance.

Step 1. Compute the probabilities $P(c_k)$ for $k = 1, \dots, p$.

Step 2. Form a table showing the conditional probabilities

$$P(f_1|c_k), \quad P(f_2|c_k), \quad \dots, \quad P(f_n|c_k)$$

for all values of $f_1, f_2, \dots, f_n$ and for $k = 1, \dots, p$.

Step 3. Compute the products

$$q_k = P(x_1|c_k)P(x_2|c_k)\cdots P(x_n|c_k)P(c_k)$$

for $k = 1, \dots, p$.

Step 4. Find j such $q_j = \max\{q_1, q_2, \dots, q_p\}$.

Step 5. Assign the class label c_j to the test instance X .

Remarks

In the above algorithm, Steps 1 and 2 constitute the learning phase of the algorithm. The remaining steps constitute the testing phase. For testing purposes, only the table of probabilities is required; the original data set is not required.

6.3.5 Example**Problem**

Consider a training data set consisting of the fauna of the world. Each unit has three features named “Swim”, “Fly” and “Crawl”. Let the possible values of these features be as follows:

Swim	Fast, Slow, No
Fly	Long, Short, Rarely, No
Crawl	Yes, No

For simplicity, each unit is classified as “Animal”, “Bird” or “Fish”. Let the training data set be as in Table 6.1. Use naive Bayes algorithm to classify a particular species if its features are (Slow, Rarely, No)?

Sl. No.	Swim	Fly	Crawl	Class
1	Fast	No	No	Fish
2	Fast	No	Yes	Animal
3	Slow	No	No	Animal
4	Fast	No	No	Animal
5	No	Short	No	Bird
6	No	Short	No	Bird
7	No	Rarely	No	Animal
8	Slow	No	Yes	Animal
9	Slow	No	No	Fish
10	Slow	No	Yes	Fish
11	No	Long	No	Bird
12	Fast	No	No	Bird

Table 6.1: Sample data set for naive Bayes algorithm

Solution

In this example, the features are

$$F_1 = \text{"Swim"}, \quad F_2 = \text{"Fly"}, \quad F_3 = \text{"Crawl"}.$$

The class labels are

$$c_1 = \text{"Animal"}, \quad c_2 = \text{"Bird"}, \quad c_3 = \text{"Fish"}.$$

The test instance is (Slow, Rarely, No) and so we have:

$$x_1 = \text{"Slow"}, \quad x_2 = \text{"Rarely"}, \quad x_3 = \text{"No"}.$$

We construct the frequency table shown in Table 6.2 which summarises the data. (It may be noted that the construction of the frequency table is not part of the algorithm.)

Class	Features									Total
	Swim (F_1)			Fly (F_2)				Crawl (F_3)		
	Fast	Slow	No	Long	Short	Rarely	No	Yes	No	
Animal (c_1)	2	2	1	0	0	1	4	2	3	5
Bird (c_2)	1	0	3	1	2	0	1	1	3	4
Fish (c_3)	1	2	0	0	0	0	3	0	3	3
Total	4	4	4	1	2	1	8	4	8	12

Table 6.2: Frequency table for the data in Table 6.1

Step 1. We compute following probabilities.

$$\begin{aligned}
 P(c_1) &= \frac{\text{No. of records with class label "Animal"}}{\text{Total number of examples}} \\
 &= 5/12 \\
 P(c_2) &= \frac{\text{No. of records with class label "Bird"}}{\text{Total number of examples}} \\
 &= 4/12 \\
 P(c_3) &= \frac{\text{No of records with class label "Fish"}}{\text{Total number of examples}} \\
 &= 3/12
 \end{aligned}$$

Step 2. We construct the following table of conditional probabilities:

Class	Features								
	Swim (F_1)			Fly (F_2)				Crawl (F_3)	
	f_1			f_2				f_3	
	Fast	Slow	No	Long	Short	Rarely	No	Yes	No
Animal (c_1)	2/5	2/5	1/5	0/5	0/5	1/5	4/5	2/5	3/5
Bird (c_2)	1/4	0/4	3/4	1/4	2/4	0/4	1/4	0/4	4/4
Fish (c_3)	1/3	2/3	0/3	0/3	0/3	0/3	3/3	0/3	3/3

Table 6.3: Table of the conditional probabilities $P(f_i|c_k)$

Note: The conditional probabilities are calculated as follows:

$$P((F_1 = \text{Slow})|c_1) = \frac{\text{No. of records with } F_1 = \text{Slow and class label } c_1}{\text{No. of records with class label } c_1} = 2/5.$$

Step 3. We now calculate the following numbers:

$$\begin{aligned} q_1 &= P(x_1|c_1)P(x_2|c_1)P(x_3|c_1)P(c_1) \\ &= (2/5) \times (1/5) \times (3/5) \times (5/12) \\ &= 0.02 \\ q_2 &= P(x_1|c_2)P(x_2|c_2)P(x_3|c_2)P(c_2) \\ &= (0/4) \times (0/4) \times (3/4) \times (4/12) \\ &= 0 \\ q_3 &= P(x_1|c_3)P(x_2|c_3)P(x_3|c_3)P(c_3) \\ &= (2/3) \times (0/3) \times (3/3) \times (3/12) \\ &= 0 \end{aligned}$$

Step 4. Now

$$\max\{q_1, q_2, q_3\} = 0.05.$$

Step 5. The maximum is q_1 and it corresponds to the class label

$$c_1 = \text{“ Animal”}.$$

So we assign the class label “Animal” to the test instance “(Slow, Rarely, No)”.

6.4 Using numeric features with naive Bayes algorithm

The naive Bayes algorithm can be applied to a data set only if the features are categorical. This is so because, the various probabilities are computed using the various frequencies and the frequencies can be counted only if each feature has a limited set of values.

If a feature is numeric, it has to be *discretized* before applying the algorithm. The discretization is effected by putting the numeric values into categories known as *bins*. Because of this *discretization* is also known as *binning*. This is ideal when there are large amounts of data.

There are several different ways to discretize a numeric feature.

1. If there are natural categories or *cut points* in the distribution of values, use these cut points to create the bins. For example, let the data consists of records of times when certain activities were carried out. The the categories, or bins, may be created as in Figure 6.3.